



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

CENTRO DE TECNOLOGIA

GRADUAÇÃO EM ENGENHARIA ELÉTRICA

MÁRIO MARQUES DE LIMA NETO

**MONITORAMENTO DE PARÂMETROS EM “SALAS LIMPAS”
USANDO O MICROCONTROLADOR ESP32**

NATAL

Agosto de 2024

MÁRIO MARQUES DE LIMA NETO

MONITORAMENTO DE PARÂMETROS EM “SALAS LIMPAS” USANDO O MICROCONTROLADOR ESP32

Trabalho de Conclusão de Curso
apresentado ao Curso de Engenharia
Elétrica da Universidade Federal do Rio
Grande do Norte como requisito parcial
para a obtenção do título de Bacharel em
Engenharia Elétrica.

Orientador: Prof. Dr. José Luiz da Silva
Junior.

NATAL

Agosto de 2024

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Lima Neto, Mário Marques de.

Monitoramento de parâmetros em "Salas Limpas" usando o Microcontrolador ESP32 / Mário Marques de Lima Neto. - Natal, 2024.

75 f.: il.

Monografia (graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia Elétrica. Natal, RN, 2024.

Orientação: Prof. Dr. José Luiz da Silva Júnior.

1. Salas Limpas - Tecnologia - TCC. 2. Monitoramento industrial -TCC. 3. ESP32 - TCC. 4. IoT - TCC. I. Silva Júnior, José Luiz da. II. Título.

RN/UF/BCZM

CDU 004.738.5:62-4

Elaborado por SARA SUNARIA DE ALMEIDA SILVA XAVIER - CRB-15/572

MÁRIO MARQUES DE LIMA NETO

MONITORAMENTO DE PARÂMETROS EM “SALAS LIMPAS” USANDO O MICROCONTROLADOR ESP32

Trabalho de Conclusão de Curso
apresentado ao Curso de Engenharia
Elétrica da Universidade Federal do Rio
Grande do Norte como requisito parcial
para a obtenção do título de Bacharel em
Engenharia Elétrica.

Orientador: Prof. Dr. José Luiz da Silva
Junior.

Aprovado em: 15 / 08 / 2024

BANCA EXAMINADORA

Prof. Dr. José Luiz da Silva Júnior

Universidade Federal do Rio Grande do Norte - Orientador

Documento assinado digitalmente
 JOSE LUIZ DA SILVA JUNIOR
Data: 19/08/2024 16:28:43-0300
Verifique em <https://validar.iti.gov.br>

Prof. Dr. Marcos Antonio Dias de Almeida

Universidade Federal do Rio Grande do Norte - Membro Interno

Documento assinado digitalmente
 MARCOS ANTONIO DIAS DE ALMEIDA
Data: 19/08/2024 13:41:03-0300
Verifique em <https://validar.iti.gov.br>

Prof. Dr. Arrhenius Vinicius da Costa Oliveira

Universidade Federal do Rio Grande do Norte - Membro Interno

Documento assinado digitalmente
 ARRHENIUS VINICIUS DA COSTA OLIVEIRA
Data: 19/08/2024 17:14:08-0300
Verifique em <https://validar.iti.gov.br>

AGRADECIMENTOS

Gostaria de expressar minha profunda gratidão a todas as pessoas que, de alguma forma, contribuíram para a realização deste trabalho e para a minha formação acadêmica. Primeiramente, agradeço à minha família e, em especial, aos meus pais, Marcio e Ana, pelo eu apoio incondicional, tanto emocional quanto financeiro, foi essencial para que eu pudesse chegar até aqui.

Agradeço bastante a Fábria, que foi muito importante na construção deste trabalho, disponibilizando seu tempo para guiar a escrita, prezando pelas normas e pelo conteúdo com tanta dedicação. Além disso, sou grato a muitas pessoas do curso e da vida que me acompanharam ao longo do caminho e me aguentaram durante essa desafiadora jornada: Adriano, Alejandro, Armando, Daniel, Diego, Felipe, Gustavo, Helma, Heráclio, João Marcelo, João Pedro, Laura, Luca, Luiz, Lyrio, Maia, Marcell, Marcelo, Procopio, Ratis, Rodrigo, Thomas e todos os outros que fizeram parte de tudo isso.

Não poderia deixar de agradecer aos meus professores do curso de Engenharia Elétrica, por suas aulas inspiradoras e pelo conhecimento transmitido ao longo destes anos e em especial, meu orientador José Luiz, por suas orientações, paciência, compreensão e por me ajudar a concluir um ciclo tão importante para minha vida. Além disso, também devo agradecer aos mestrandos, doutorandos e demais colegas de curso que deram suporte necessário para a continuidade desta trajetória, muitos passaram por mim e tiveram impacto em minha formação, tanto acadêmica quanto comportamental.

RESUMO

O desenvolvimento de um projeto de sistema de monitoramento de baixo custo a nível industrial foi a proposta apresentada como probabilidade de garantia de eficiência e de eficácia a tal ponto que pode ser implementado na utilização das salas limpas. Este sistema consiste na aplicação da internet das coisas através do microcontrolador Esp32, diversos sensores e ThingSpeak como plataforma *online*. Tendo em vista a aplicação industrial, foi levado em consideração parâmetros como temperatura, umidade, pressão ambiente e partículas no ar, que interferem diretamente no desempenho de máquinas e seu funcionamento; tal como a durabilidade de vida útil estendida, não apresentando falhas ou interrupções. As salas limpas são ambientes controlados que são regulamentadas conforme a NBR 13413, isto é, onde sistemas de ar, filtragem e procedimentos visam manter baixos níveis de partículas no ar. As salas limpas estão sendo cada vez mais utilizadas visando melhoria de qualidade em diversos ambientes, especialmente na área da nutrição e saúde; como na manipulação de medicamentos e no armazenamento de equipamentos cirúrgicos.

Palavras-chave: Salas Limpas. Monitoramento. Esp32. IoT.

ABSTRACT

The development of a low-cost monitoring system project at an industrial level was presented as a probability of guaranteeing efficiency and effectiveness to such an extent that it could be implemented in the use of clean rooms. This system consists of applying the internet of things through the Esp32 microcontroller, various sensors and ThingSpeak as an online platform. With an industrial application in mind, issues such as temperature, humidity, ambient pressure and particles in the air were taken into consideration, which directly interfere with the performance of the machines and their operation; such as the durability of endless service life, without failures or interruptions. Clean rooms are controlled environments that are regulated by ISO standards, that is, they have quality and safety certification for their operation. Clean rooms are being increasingly used, changing quality improvements in various environments, especially in healthcare; such as handling medications and storing surgical equipment.

Keywords: Clean Rooms. Monitoring. Esp32. IoT.

LISTA DE ILUSTRAÇÕES

Figura 1-	Aumento da quantidade de dispositivos conectados à internet.....	18
Figura 2-	Análise de Big Data com <i>IoT</i>	20
Figura 3-	Pinagem do Esp32-Wroom-32 (Visão Superior).....	23
Figura 4-	Visão superior e inferior da placa Esp32.....	24
Figura 5-	Representação das ligações do DHT11 com o microcontrolador...	25
Figura 6-	Foto do sensor DHT11.....	26
Figura 7-	Representação da ligação do Bmp180 ao microcontrolador.....	27
Figura 8-	Foto do sensor BMP180.....	28
Figura 9-	Visão superior e inferior do sensor MQ-135.....	29
Figura 10-	Visão superior e inferior do sensor DSM501a.....	30
Figura 11-	Conjunto de pulsos do DSM501a para calcular o <i>Low Ratio</i>	31
Figura 12-	Representação em blocos da arquitetura do módulo.....	32
Figura 13-	Interface do Arduino IDE.....	33
Figura 14-	Exemplo de sala limpa vista de fora.....	36
Figura 15-	Concentração de partículas por m ³ segundo a ISO 14644-1.....	37
Figura 16-	Representação da arquitetura do projeto.....	38
Figura 17-	Circuito montado do DHT11.....	40
Figura 18-	Código referente ao sensor DHT11.....	40
Figura 19-	Código das funções <i>Temperatura()</i> e <i>Umidade()</i>	41
Figura 20-	Leitura do sensor de temperatura e umidade após uma hora.....	42
Figura 21-	Condições meteorológicas segundo o <i>weather.com</i>	42

Figura 22-	Circuito montado com o BMP180.....	43
Figura 23-	Código de inicialização do sensor BMP180.....	44
Figura 24-	Código de leitura e exibição da pressão ambiente.....	44
Figura 25-	Medição da pressão pelo sensor BMP180.....	45
Figura 26-	Curva da pressão atmosférica ao longo do dia 17/07.....	45
Figura 27-	Circuito montado do MQ-135.....	46
Figura 28-	Código de inicialização do sensor MQ-135.....	47
Figura 29-	Código da medição do sensor MQ-135.....	47
Figura 30-	Curva dos valores medidos pelo sensor MQ-135.....	48
Figura 31-	Curva de valores medidos pelo sensor MQ-135 à exposição.....	48
Figura 32-	Circuito montado do DSM501a.....	49
Figura 33-	Código do sensor DSM501a para definição das variáveis.....	50
Figura 34-	Código do sensor DSM501a para cálculo da concentração.....	51
Figura 35-	Medidas do DSM501a no monitor serial.....	51
Figura 36-	Circuito inteiro montado no ambiente doméstico.....	54
Figura 37-	Configurações do canal no Thingspeak.....	55
Figura 38-	Código da configuração do canal.....	56
Figura 39-	Código de atualização dos valores no ThingSpeak.....	57
Figura 40-	Circuito montado realizando medições no ambiente industrial...	58
Figura 41-	Curva dos valores medidos no ThingSpeak no ambiente residencial.....	60
Figura 42-	Gráficos de temperatura, umidade e pressão do dia 28/07/2024.	61
Figura 43-	Curva dos valores medidos no ThingSpeak no ambiente industrial.	62

Figura 44- Gráficos de temperatura, umidade e pressão dos dias 29 e 30/07/2024..... 63

LISTA DE TABELAS

Tabela 1- Portas de conexão dos sensores com Esp32.

Tabela 2- Custos para montagem do sistema de monitoramento.

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ANVISA	Agência Nacional de Vigilância Sanitária
API	Application Programming Interface
BMP	Barometric Pressure
C/C++	C-Plus-Plus
CO	Monóxido de Carbono
CO ₂	Gás Carbônico
DHT	Digital Humidity and Temperature
DSM	Dust Sensor Module
g	Grama
GHz	Gigahertz
GPIO	General Purpose Input/Output
HDFS	Hadoop Distributed File System
hPa	Hectopascal
IDE	Interface Development Environment
ISO	Organização Internacional de Normalização
kΩ	Kiloohm
kB	Kilobyte
L	Litro
m	Metro
MHz	Megahertz
NBR	Norma Brasileira Regulamentadora
NH ₃	Amônia
Pa	Pascal
RC	Resistor-Capacitor
ROM	Read-Only Memory
SEBRAE	Serviço Brasileiro de Apoio às Micro e Pequenas Empresas
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
TCP	Protocolo de Controle de Transmissão
UFRN	Universidade Federal do Rio Grande do Norte

V	Volt
Vcc	Voltagem em Corrente Contínua
VIN	Input Voltage
Wifi	Wireless Fidelity
°C	Graus Celsius
µm	Micrômetro

SUMÁRIO

1 INTRODUÇÃO.....	14
1.1 Motivação.....	14
1.2 Objetivo.....	15
1.3 Estrutura.....	16
2 FUNDAMENTAÇÃO TEÓRICA.....	17
2.1 Internet das Coisas.....	17
2.2 Internet das Coisas Industrial.....	18
2.3 Sistemas Embarcados e Microcontroladores.....	20
2.4 Microcontrolador Esp32.....	22
2.5 Sensor de Umidade e Temperatura.....	25
2.6 Sensor de Pressão Ambiente.....	26
2.7 Sensor de Qualidade do Ar.....	28
2.8 Sensor de Poeira.....	30
2.9 Arduino IDE.....	33
2.10 ThingSpeak.....	34
2.11 Salas Limpas.....	35
3 METODOLOGIA.....	37
3.1 Arquitetura do Projeto.....	37
3.2 Testes Individuais.....	39
3.2.1 Sensor de Temperatura e Umidade.....	39
3.2.2 Sensor de Pressão.....	42
3.2.3 Sensor de Qualidade do Ar.....	46
3.2.4 Sensor de Poeira.....	48
3.3 Teste em Conjunto.....	52
3.3.1 Configuração da Montagem.....	52

3.3.2 Integração ao ThingSpeak.....	54
3.3.3 Testes Propostos.....	57
4 RESULTADOS E DISCUSSÕES.....	59
4.1 Resultados Obtidos.....	59
4.2 Discussões.....	63
5 CONSIDERAÇÕES FINAIS.....	66
6 REFERÊNCIAS.....	68
7 APÊNDICE.....	71

1 INTRODUÇÃO

1.1 Motivação

A tecnologia vem avançando com maior velocidade nas últimas décadas, suprimindo uma crescente demanda que fomenta a inovação e a criatividade para a resolução de problemas cada vez mais complexos. Esse avanço no que se diz respeito à parte física vem por meio de descobertas ao longo de séculos de história. Porém, desde o surgimento da chamada Indústria 3.0, em que os primeiros controladores programáveis revolucionaram a automação industrial, a questão de programas e computação caminharam em passos essenciais para a obtenção da sofisticação alcançada pela revolução tecnológica, alcançada na revolução tecnológica presente a partir dos anos 2000.

Tendo em vista esses avanços, uma gama maior de possibilidades abre caminho para facilitar processos em diversas áreas e torná-los mais eficientes. Por exemplo, segundo o site do Sebrae (2023), o papel dos sensores tem se tornado fundamental em aplicações industriais, sendo importantes para contagem de materiais, nível de fluidos, controle de direção e entre outros. Dessa forma, o uso de sensores tornam os processos mais rápidos e diminuem as perdas materiais. Porém, não é apenas a questão material que há uma preocupação em diminuir as falhas, mas, também, a parte da segurança dos trabalhadores humanos, os quais ficam sujeitos a acidentes de trabalho ao manusear aparatos perigosos. Em decorrência disso, várias máquinas utilizadas em indústrias com um alto grau de qualidade possuem sensores que percebem a presença de partes humanas em um certo limite e mandam um sinal de desligamento caso essa barreira seja ultrapassada.

Dentro de um universo de aplicações industriais, pode-se analisar o uso do monitoramento de diversas variáveis em uma sala limpa para fabricação de medicamentos. De acordo com matérias publicadas pela Asmontec (2018), uma empresa especializada na construção de salas limpas, o controle minucioso da pureza do ambiente é imprescindível para evitar a contaminação dos produtos. Aspectos como: Fluxo de ar, temperatura, pressão, ruído, iluminação e contaminação por partículas são monitorados para reduzir a introdução, geração e a retenção de contaminantes no interior da sala limpa. Dessa forma, ao possuir um sistema de controle e monitoramento eficiente, é possível manter os níveis de

pureza do ambiente dentro das normas técnicas da ABNT, normas legislativas da Anvisa e da vigilância sanitária.

Além do mais, em relação à questão da importância da segurança em ambientes industriais, incêndios em indústrias do Brasil são um acontecimento recorrente. Este fenômeno ocorre, em sua maioria, devido à forte presença de itens inflamáveis, a falta de manutenção em equipamentos e em maquinários que podem produzir faíscas e problemas nas instalações elétricas. Tendo isso em vista, é possível notar que a maior parte dos incêndios em indústrias são causados devido à falta de monitoramento de fatores muitas vezes negligenciados, por não afetarem diretamente a atividade fim destas indústrias, porém, causadoras de um risco capaz de provocar prejuízos significativos quando a conflagração acontece.

De acordo com uma pesquisa realizada pelo Instituto Sprinkler do Brasil (ISB) realizada no ano de 2021, foram contabilizados 2.301 incêndios estruturais no País. Tendo em vista esse número alarmante, os locais mais frequentes e as causas já mencionadas, é possível entender que uma maior fiscalização e monitoramento de possíveis falhas e elementos perigosos ajudariam a mitigar essas tragédias que causam uma grande perda material e põem vidas em risco.

1.2 Objetivo

Tendo à disposição o conhecimento acerca da Internet das Coisas e diversos dispositivos eletrônicos a fim de auxiliar no recolhimento de dados, em testes e consulta, este trabalho propõe o desenvolvimento de um sistema capaz de realizar o monitoramento em tempo real de grandezas específicas que sirvam em aplicações industriais de fácil visualização e consulta, para tornar mais prático o monitoramento desses ambientes. Outro ponto é a intenção de realizar os testes pensando no ambiente residencial e industrial, para propor o uso do sistema como forma de compor o monitoramento de salas limpas com baixo custo. Além disso, outro objetivo é informar sobre os equipamentos específicos utilizados, de forma que essas informações sejam esclarecedoras para o uso individual destes, para futuros projetos envolvendo microcontroladores programados, sensores e a Internet das Coisas.

1.3 Estrutura

Este trabalho está estruturado da seguinte maneira; A introdução apresenta a visão geral do estado da arte que possibilita o sistema de monitoramento com o Esp32, como um microcontrolador utilizado como recurso tecnológico avançado principalmente para aplicação industrial, especialmente nas salas limpas.

O segundo capítulo diz respeito à internet das coisas, sua aplicabilidade e a sua importância na automação industrial, além de discorrer sobre os equipamentos usados na aplicação em questão e seu contexto acerca dos avanços tecnológicos.

Já o terceiro capítulo, encontra-se dividido com subitens de montagem e teste, para cada um dos sensores utilizados no sistema integrado à plataforma *online*, focando na validação por experimentações individuais e o teste em conjunto do sistema proposto.

Em relação ao quarto capítulo, os resultados dos testes são analisados e as vantagens e possíveis aplicações são discutidas, se embasando na proposta de sistema teorizado e testado, destacando a utilização em conjunto e pensando no mercado atual.

E, finalmente, o Capítulo 5 apresenta reflexões acerca do trabalho como um todo, abordando possíveis melhorias e sugestões propostas levando em consideração todo o conhecimento teórico reunido e a experimentação feita por meio dos equipamentos utilizados.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Internet das Coisas

Primeiramente, antes de descrever sobre a composição dos dispositivos e seus funcionamentos, é necessário explicar sobre o conceito da Internet das Coisas (*Internet of Things*, em inglês). Segundo Rose, Eldridge e Chapin (2015, p. 7), esse termo foi utilizado para descrever sistemas em que cujos objetos no mundo físico têm a capacidade de se conectar à internet por meio de sensores, em 1999. Naquela época, o termo foi criado para uma aplicação com *tags* com tecnologia de *Radio-Frequency Identification* (RFID) para contabilizar e rastrear mercadorias de forma mais prática. Já nos dias de hoje, esse conceito ganhou cada vez mais popularidade conforme o avanço da tecnologia abriu mais possibilidades.

Tendo isso em vista, é possível afirmar que a junção de diversos componentes físicos com a conexão à internet ajudam a resolver diversos problemas encontrados no dia-a-dia, tanto em ambientes industriais como em ambientes residenciais. Pois, esses dispositivos automatizados com integração à rede e entre si podem estabelecer uma comunicação eficiente, abrindo um leque de possibilidades para quem desenvolve esses sistemas e para a experiência do consumidor que as utiliza.

O uso da *IoT* está aumentando ao longo dos anos e se beneficiar desse ponto definirá como as atividades econômicas serão realizadas no futuro. De acordo com dados da consultoria *IoT Analytics* coletados em 2023, mais de 14,4 bilhões de dispositivos tinham conexão com a internet até o final de 2022, como no gráfico da Figura 1, e realizou-se uma projeção de que esse número crescerá para cerca de 27 bilhões até 2025. Como consequência da maior quantidade de aparelhos conectados, tem-se a integração de diversos dispositivos com funções especializadas coletando dados para melhorar suas aplicações e ampliando suas funcionalidades.

Figura 1 - Aumento da quantidade de dispositivos conectados à internet



Fonte: Poder 360. 2020. Disponível em:

<https://www.google.com/url?q=https://www.poder360.com.br/brasil/dispositivos-iot-de-vem-chegar-a-27-bilhoes-ate-2025/&sa=D&source=docs&ust=1723501573534272&u sg=AOvVaw1qIshFT5gglGONGS0mu8ID>. Acesso em: 24 de abril de 2024.

Um exemplo a ser visto é a empresa Tramontina, a qual desenvolveu um fogão conectado à internet que promete automatizar diversos processos, como controle de temperatura, tempo e entre outros. Esses processos serão administrados e consultados por meio de um aplicativo da própria companhia que realiza essa integração com o “fogão *IoT*”, que servirá como um livro de receitas exibindo vídeos, dicas de diversas receitas e configuram o próprio fogão como o usuário desejar.

2.2 Internet das Coisas Industrial

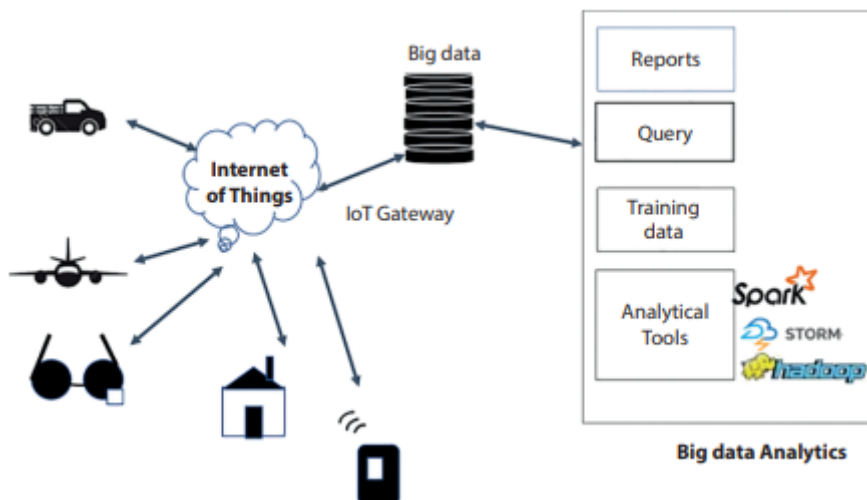
O termo, em inglês, *Industrial Internet of Thing (IIoT)* é autoexplicativo e indica a utilização da já mencionada Internet das Coisas para aplicações industriais. Essas

aplicações podem ocorrer por meio do uso de sensores inteligentes, atuadores elétricos e entre outros dispositivos com o objetivo de tornar os processos industriais mais eficientes e confiáveis. Por certo, essas qualidades são importantes para esse tipo de aplicação, pois ter informações constantes ajudam também o estado do ambiente e das máquinas, com o objetivo evitar acidentes e manter o funcionamento do ambiente e dos equipamentos.

Para uma maior compreensão de como esse escopo engloba várias possibilidades, de acordo com Anandan, Surendran, et al (2022, p. 92-95), alguns fundamentos da *IIoT* conseguem condensar seus recursos principais e mais utilizados dentro das aplicações industriais. A primeira delas se trata da computação por meio da nuvem, em que o processamento, armazenamento e autenticação ocorrem em servidores que promovem uma tecnologia de computação distribuída sob demanda. Em consequência disso, é possível prover *softwares* autenticados por esse método e armazenar e processar dados via internet, serviços providos, uma grande parte dos casos, por meio do serviço de nuvem do Google.

Outro fundamento a ser destacado é a análise de *Big Data*, pois, ao trabalhar com uma quantidade elevada de sensores e dispositivos conectados via internet, o grande número de dados coletados exige que haja um pré-processamento e correlação com o objetivo de utilizar apenas os dados possivelmente relevantes para as medições. Essa avaliação de dados é atualmente feita por ferramentas de agrupamento, classificação e predição, como o *MapReduce*, *HDFS* e o *Spark*.

Ao retornar para as características da própria Internet das Coisas, a qual mudou como as atividades industriais são feitas, ainda há muito a se explorar. A *IoT* permite uma organização de dispositivos físicos ou virtuais identificados que torna possível uma intercomunicação de forma direta via internet, ou seja, sem que haja a necessidade de pessoas na operação, como a representação na Figura 2. Exemplos de dispositivos com esse tipo de sistema são os *smartwatches*, veículos *smarts*, *Alexas* e aparelhos de monitoramento de saúde remotos, todos utilizam aplicações de *IoT* em tempo real e realizam um pré-processamento de informações via nuvem, como citado anteriormente.

Figura 2 - Análise de Big Data com *IoT*

Fonte: Industrial Internet of Things (IIOT). 2020.

2.3 Sistemas Embarcados e Microcontroladores

Ao descrever sistemas computacionais, é possível realizar uma separação entre duas categorias distintas. Uma delas é o “computador de mesa”, o qual é visualizado logo quando se fala de computação. Porém, o segundo tipo é mais abundante no dia a dia do que o primeiro e se trata dos computadores embarcados, ou melhor, os sistemas embarcados. De acordo com John Catsoulis (2003, p. 3-4), os sistemas embarcados são computadores que estão integrados em outros sistemas com o propósito de controle ou monitoramento. A arquitetura subjacente dos dois tipos é fundamentalmente parecida: Processador, memória e formas de entrada e saída.

A principal diferença entre computadores de mesa e os computadores embarcados está nas funcionalidades de uso. Em primeiro lugar, sua versão de mesa pode rodar uma variedade de programas, com recursos orquestrados por um sistema operacional, realizando tarefas que vão de executar músicas até gerenciar banco de dados conectados à internet, ou seja, sua funcionalidade é generalista. Por outro lado, os embarcados normalmente são dedicados a tarefas específicas, tomando proveito de microprocessadores com sistemas definidos por *software* (a programação) e, não, por *hardware* (o componente físico). Além do mais, computadores embarcados não precisam, necessariamente, de um sistema

operacional e, normalmente, possuem uma aplicação que roda permanentemente, contido em um sistema de memória não volátil.

Para entender sobre o funcionamento dos sistemas embarcados, é importante citar sobre os microcontroladores. De acordo com Kerschbaumer (2018, p. 10-12), os microcontroladores são circuitos integrados, projetados para desempenhar uma variedade de funções computacionais com um único chip, dependendo apenas de uma fonte de alimentação externa. Esses chips abrigam uma unidade central de processamento (CPU) e uma gama de periféricos essenciais para suas operações, como memória de dados, memória de programa e circuito de *clock*.

A principal vantagem dos microcontroladores reside na integração desses periféricos em um único componente, simplificando sua utilização e reduzindo custos de produção. Além disso, sua versatilidade é destacada pela capacidade de adaptar seu comportamento de acordo com o *software* carregado, permitindo uma ampla gama de aplicações com um único dispositivo e tendo a capacidade de seu tamanho variante facilitar o acoplamento físico em equipamentos com dimensões variáveis.

Esse tipo de controlador encontra aplicações em uma vasta gama de produtos, desde centrais de alarme e teclados de computador até relógios de pulso, eletrodomésticos e veículos. Além do mais, na indústria, esses componentes desempenham um papel importante em controladores lógicos, sensores inteligentes, inversores, interfaces homem-máquina e entre outros dispositivos e sistemas automatizados.

Indo além de aplicações estritamente comerciais, existe a placa e a plataforma de prototipagem chamada de Arduino, voltada para o desenvolvimento de projetos eletrônicos. Possuindo um microcontrolador capaz de executar programas e verificar a qualidade de entradas e saídas, fora ser *open-source*, essa placa é bastante versátil e muito utilizada no meio acadêmico, tanto para o ensino de programação voltada para eletrônica quanto para a prototipação.

Seu *software* é programado por meio de uma linguagem derivada do C/C++, possui uma gama de módulos acopláveis com funcionalidades diferentes e compatíveis com o seu sistema, como sensores, motores e botões. Dessa forma,

esses periféricos contribuem ao adicionar funcionalidades as quais o Arduino sozinho não conseguiria contemplar. Por outro lado, existem várias opções de placas no mercado, que atendem múltiplas aplicações, incluindo elementos não incluídos no Arduino por fábrica. Por exemplo, a placa Esp32 possui conexão *Wifi* e *Bluetooth*, aumentando o leque de possibilidades de aplicações.

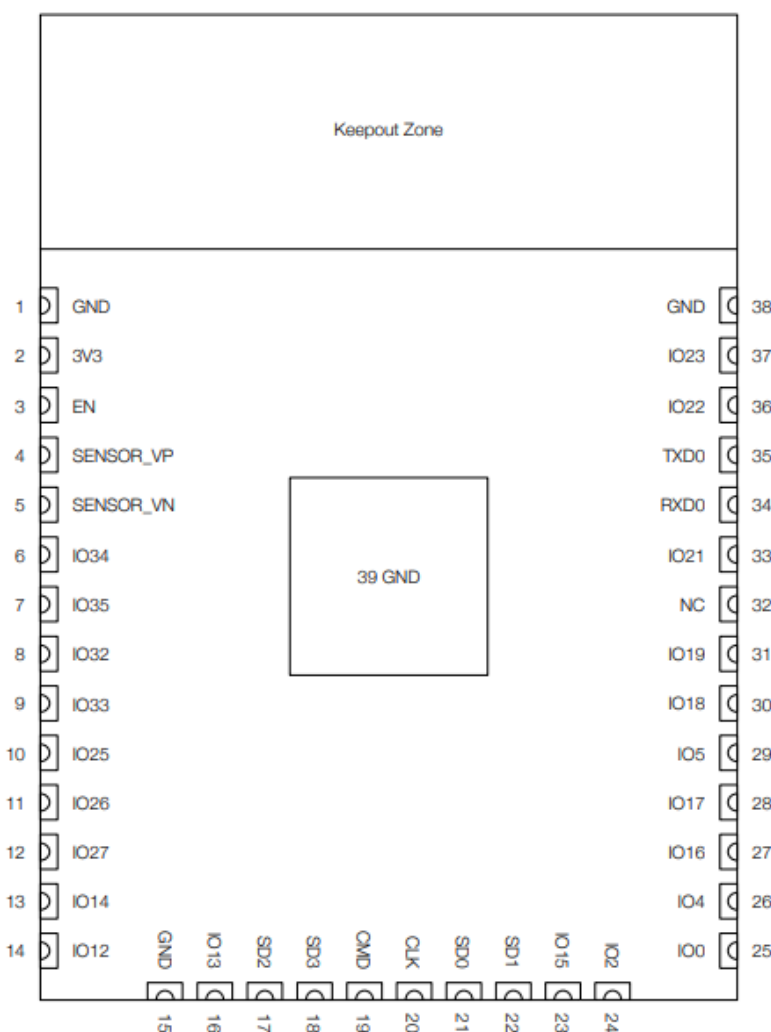
2.4 Microcontrolador Esp32

O Esp32 é um microcontrolador versátil e potente desenvolvido pela empresa *Espressif Systems* e é amplamente utilizado em diversas aplicações que variam de simples dispositivos com *IoT* até sistemas embarcados complexos. Esse microcontrolador possui recursos e capacidades importantes para torná-lo bastante eficiente, como por exemplo: A arquitetura do Esp32 se baseia na utilização do microprocessador *Xtensa® single-/dual-core 32-bit LX6* e a inclusão de diversos recursos de manuseio da fonte de energia própria e uma lista grande de periféricos. Além disso, seu *design* comporta funcionalidades de Wi-Fi e Bluetooth com 2.4 GHz com um chip semiconductor tailandês de *low-power* e 40 nm.

Sobre a questão do processamento e da memória, o Esp32 pode atuar no modo de *single-core* (com 2.10 *CoreMark*/MHz) ou *dual-core* (com 4.14 *CoreMark*/MHz), possui 448 kB de *ROM* e 520 kB de *SRAM* e possuindo osciladores RC e de 8 MHz internos com calibração, tendo espaço e processamento o bastante para muitas aplicações. Outro aspecto bastante importante e que possibilita conectar os diversos periféricos ao *Esp32* é a quantidade de portas lógicas, possuindo 34 portas *GPIO* programáveis que incluem pinos de *reset* e *startup*, de entrada e saída e pinos para acesso direto à memória *flash*.

Tendo isso em vista, o modelo escolhido para o projeto é o Esp32-WROOM-32, o qual é uma categoria mais acessível de microcontrolador, mas com uma eficiência comparada ao modelo padrão. Ele também tem portas que têm a tensão recomendada de 3,3 V, diferenciando-se da placa Arduino (que usa 5 V). Essas portas possuem diversas funções e é importante ter noção de quais devem ser utilizadas, pois algumas têm aplicações específicas, as quais podem ser vistas na Figura 3.

Figura 3 - Pinagem do Esp32-Wroom-32 (Visão Superior)



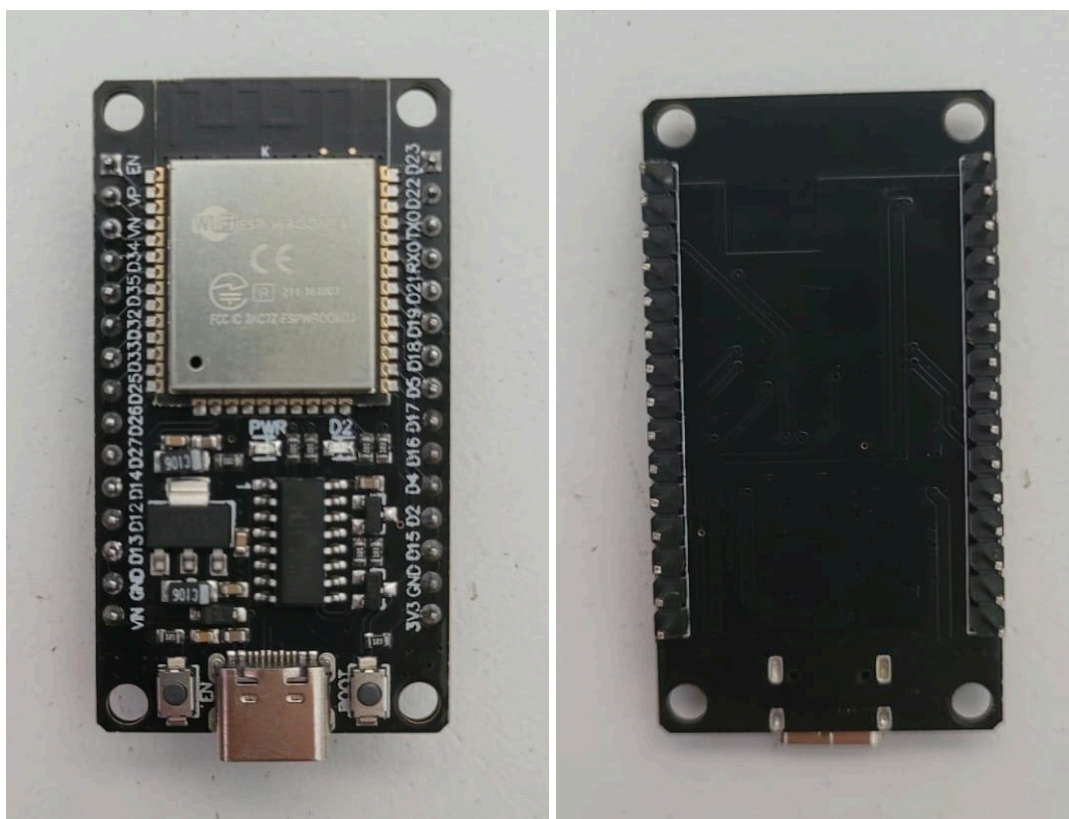
Fonte: Datasheet Esp32 Wroom. 2024.

Quando se analisa a pinagem mais à fundo do Esp32, é possível ver algumas diferenciações, de acordo com o *Datasheet* do referido microcontrolador (ESPRESSIF, 2024). Primeiramente, os 15 pinos *ADC* (*Analog to Digital Converter*) fazem parte de um circuito que converte uma tensão de entrada analógica para uma equivalente digital, tendo dois tipos distintos de conversores: O *ADC1*, que possui 8 canais, e o *ADC2*, com 10 canais, ambos tendo uma resolução de 12 *bits*, o que varia os valores obtidos de 0 até 4095. Um detalhe importante sobre esse circuito é que o conversor *ADC 2* é diretamente ligado ao módulo *Wi-fi*, impossibilitando leituras enquanto a conexão sem fio estiver ativa. Já o *ADC1* pode sofrer com ruídos durante o uso do *Wi-fi*.

Além do mais, o *Esp32* possui 2 pinos que realizam o trabalho contrário ao anterior, são *DAC* (*Digital to Analog Converter*), ou seja, convertem um sinal digital para analógico de tensão, porém, apenas com a resolução de 8 *bits*. Apesar dos pinos serem semelhantes à primeira vista, as portas *GPIO* 6 até *GPIO* 11 são pinos que se conectam diretamente com a memória Flash SPI integrada do *Esp32*, então elas não serão usadas.

Também é possível notar que existem pinos importantes, mas não fazem parte do *GPIO*, como o pino *VIN*, o qual é uma entrada de alimentação com a faixa de tensão recomendada de 7 a 12 V, dois pinos *GND* (*Ground*), ligados a terra, o pino *EN* (*Enable*), sendo apenas um pino de ativação do regulador de 3,3 V, e o pino de 3,3 V, que será utilizado para alimentar os periféricos ligados ao próprio *Esp32*. Assim, a foto das visões superior e inferior está na Figura 4.

Figura 4 - Visão superior e inferior da placa *Esp32*.



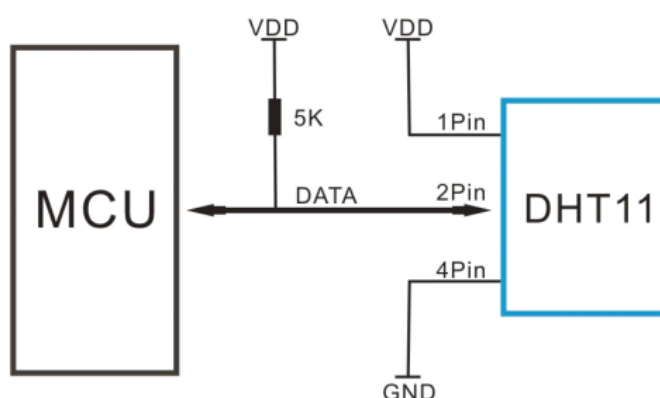
Fonte: De Autoria própria (2024).

2.5 Sensor de Umidade e Temperatura

O sensor de temperatura e umidade escolhido foi o *DHT11*, pois, além de integrar estas duas informações em um único periférico e ter um baixo custo financeiro, ele possui uma estabilidade a longo prazo em suas leituras. O *DHT11* realiza a medição da temperatura por meio de um componente *NTC* (*Negative Temperature Coefficient*), ou seja, a diminuição da resistência deste é inversamente proporcional ao aumento da temperatura. A umidade relativa também é medida por meio da variação da resistência do componente. Assim, os dados são mandados para um microcontrolador com 8 bits, para gerar uma resposta rápida, com baixa interferência e boa qualidade na informação.

Também segundo o *datasheet Mouser* (2024, p. 2-5), o *DHT11* possui 4 pinos, os quais são conectados de acordo com as necessidades do usuário. Existe um para a alimentação *Vcc*, que será alimentado com 3,3 V, um para o *GND*, um pino de dados, que devem ser ligados à alimentação do *Esp32* por meio de um resistor de 5 k Ω para garantir uma maior segurança às portas de leitura do microcontrolador, e um último pino que não é utilizado. Estes pinos estão representados na Figura 5 e na Figura 6. Já pelo lado do software, o *DHT11* possui uma biblioteca própria com funções prontas para acessar as leituras de uma forma mais intuitiva, o que facilita na parte da montagem e programação.

Figura 5 - Representação das ligações do *DHT11* com o microcontrolador.

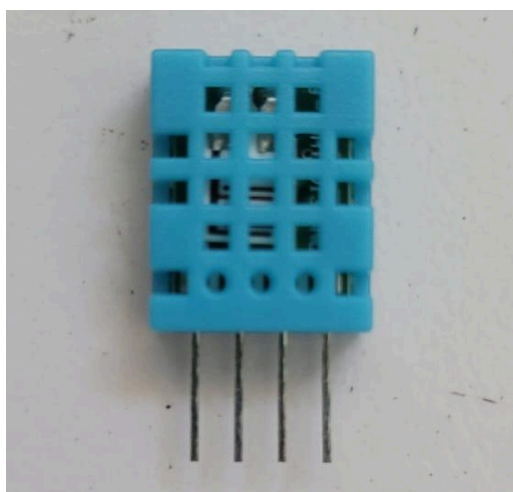


Fonte: *DHT11 Humidity & Temperature Sensor*. 2024.

O sensor é capaz de medir temperaturas entre 0 e 50 °C, além de uma umidade relativa do ar de 20 a 90%. Isso com uma precisão de 2 °C e 5% para mais

ou para menos, respectivamente. Esse controle de temperatura e da umidade do ambiente é importante em diversas aplicações. É possível citar, por exemplo, a empresa farmacêutica, que precisa monitorar ativamente esses parâmetros no ar. De acordo com Vital, Braga (2019, p. 11), os componentes termolábeis são medicamentos sensíveis à temperatura do ambiente, podendo alterar sua eficácia caso sejam manipulados ou armazenados em locais sem a temperatura adequada, a qual é entre 2 e 8 °C. Além de tudo, o INTS (Instituto Nacional de Tecnologia e Saúde) define que a umidade relativa do ar recomendada para isso é entre 40 e 70%.

Figura 6 - Foto do sensor DHT11



Fonte: De Autoria própria (2024).

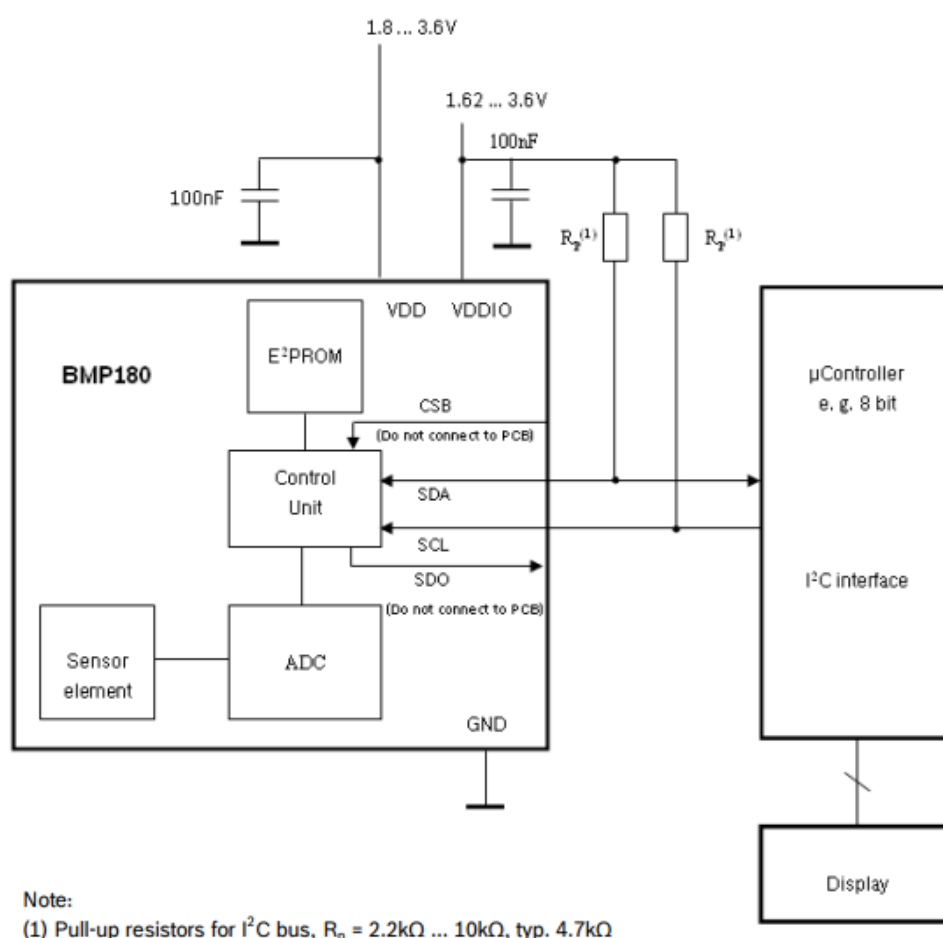
2.6 Sensor de Pressão Ambiente

Para a medição da pressão ambiente, o sensor *Bmp180* foi escolhido. Apesar dele ser capaz de medir a pressão atmosférica em Pa (Pascal) e a temperatura, apenas a primeira função será utilizada. De acordo com Britto (2018, p. 55), é importante que haja um controle da pressão ambiente sobre as salas, não de uma forma absoluta, mas quanto à pressão diferencial entre elas. Com outras palavras, a sala limpa precisa ter uma pressão estática maior que os ambientes adjacentes, pois, dessa forma, as partículas presentes no ar das salas sujas não entrarão nas salas limpas.

Esse sensor possui um semicondutor sensível à flutuação da pressão do ar, ou seja, é de um material cuja resistência varia de acordo com a pressão do ar, como o silício. Dessa forma, o material conduzirá mais corrente conforme o valor do parâmetro já mencionado, possibilitando sua medição. Tendo isso em vista, a circuitagem interna do sensor é representada na Figura 7.

Sendo assim, a pressão ambiente de uma sala para outra deve ser entre 5 a 20 Pa. Essa medição pode ser realizada pelo sensor *Bmp180*, segundo o *datasheet* da *Bosch* (2013, p. 2-5), pois seu alcance de medição é de 300 a 1100 hPa nas condições de temperatura entre 0 e 60 °C, o que atende às condições esperadas, tendo uma precisão absoluta de $\pm 2,5$ hPa. O sensor em questão possui quatro pinos de conexão para ligar ao *Esp32*, eles são: Pino Vcc de 3,3 V, um pino *GND* e os pinos *SDA* e *SCL*.

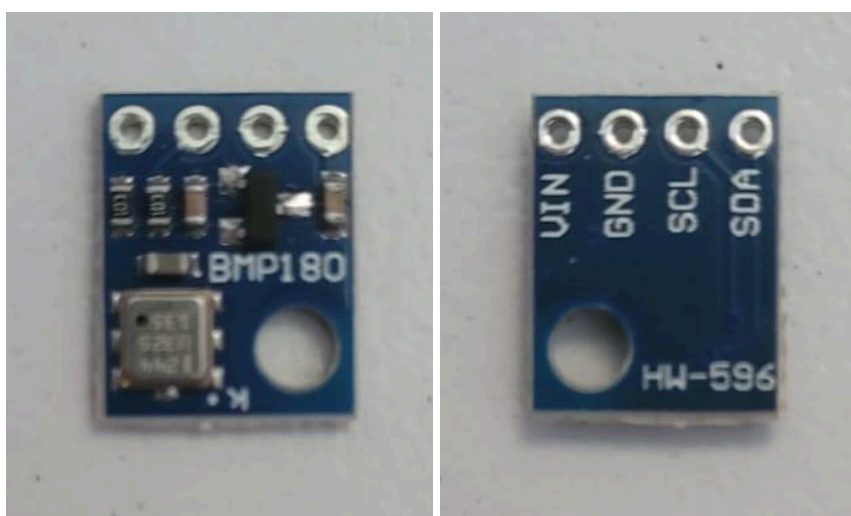
Figura 7 - Representação da ligação do Bmp180 ao microcontrolador



Fonte: BMP180 Digital Pressure Sensor. 2013.

Para a programação no *Esp32*, será utilizado a biblioteca *Adafruit Bmp085*, apesar dela ser feita pensada no sensor *Bmp085*, o qual também mede temperatura e pressão, esta biblioteca também é compatível com o *Bmp180*. Na parte das conexões, o pino *SCL* (“*Serial Clock*”) será conectado à porta que servirá como *clock* do microcontrolador do sensor e o pino *SDA* (“*Serial Data*”) emitirá dados de saída dos valores de pressão e temperatura ambientes que são lidos pelo *Esp32*. Porém, como já foi mencionado, apenas a pressão será lida, pois apenas o sensor *DHT11* irá medir a temperatura, para evitar possíveis redundâncias. Além do sensor ser ligado diretamente ao *Esp32* por meio dos jumpers, os pinos *SCL* e *SDA* devem ser ligados à trilha de 3,3 V por meio de um resistor cada, com o valor entre 2,2 e 10 k Ω , para dar mais segurança, como na Figura 8.

Figura 8 - Foto do sensor BMP180



Fonte: De Autoria própria (2024).

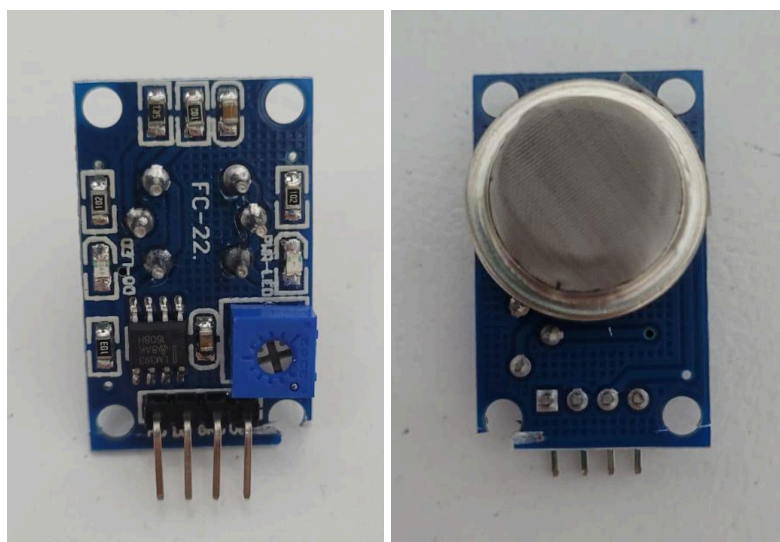
2.7 Sensor de Qualidade do Ar

Por fim, mais um parâmetro a ser monitorado é a qualidade do ar da sala em questão. Esse controle deve ser feito pois a presença de certas partículas indesejadas comprometem a realização de procedimentos cujos cuidados quanto à pureza devem ser a maior possível. Ainda utilizando o artigo de Britto (2018, p. 46), as salas limpas necessitam de procedimentos para monitorar as concentrações de

partículas em suspensão no ar, atendendo aos níveis apropriados de limpeza conforme definido pelo usuário e de acordo com as normas técnicas vigentes.

Tendo isso em vista, o sensor escolhido foi o *MQ-135*, fabricado pela empresa chinesa Huawei, e que é capaz de detectar a presença de vários gases diferentes no ambiente, entre eles: CO_2 , CO , álcool, NH_3 , Tolueno, Acetona e fumaça. Ao realizar o projeto da sala limpa, é necessário ter uma noção do tipo de serviço que será realizado e quais tipos de partículas podem ser expelidas no ar, caso seja necessária uma averiguação mais precisa das partículas presentes no ar. Conforme o *datasheet* da empresa *Hanwei*, referente ao *MQ-135*.

Figura 9 - Visão superior e inferior do sensor *MQ-135*



Fonte: Autoria própria (2024).

Ao considerar a perspectiva superior, da esquerda para a direita, os pinos são: O pino da saída analógica, o pino da saída digital, o pino *GND* e o *Vcc*, estes dois últimos presentes em outros sensores já explicados, como na Figura 9. A alimentação por meio do *Vcc* deve ser realizada utilizando a tensão entre 2,5 e 5 V e, segundo o *datasheet*, é importante que o *MQ-135* passe por um período de aquecimento antes do uso e quanto mais tempo ele passar desligado, mais tempo será necessário para esse processo antes do uso. O tempo de aquecimento recomendado é de, pelo menos, 24 horas. É importante salientar que o tempo prolongado sem uso degrada os componentes internos e afetam sua sensibilidade,

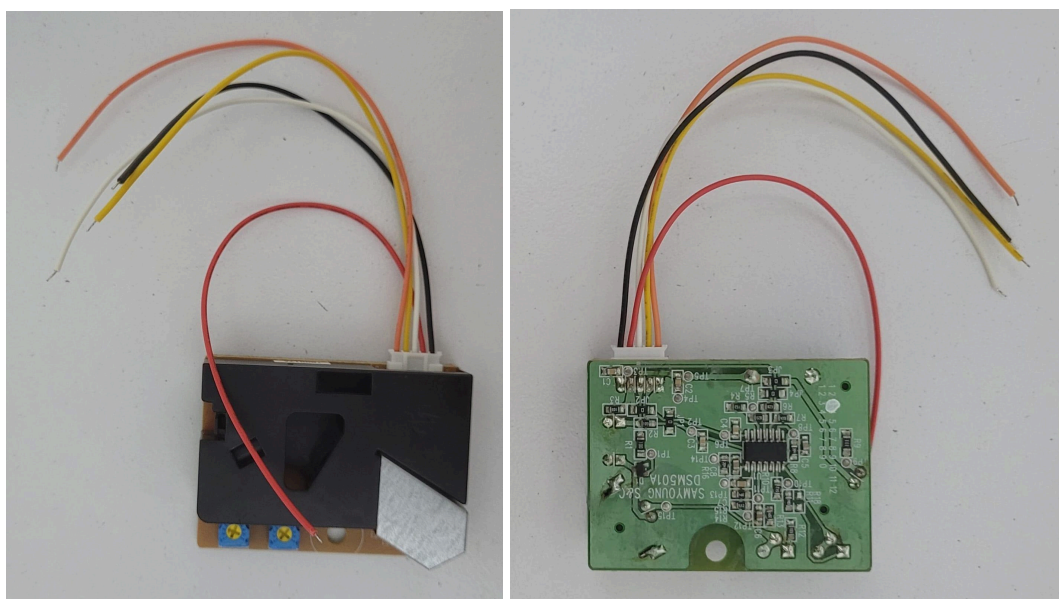
necessitando de maiores tempos de aquecimento para manterem uma estabilidade confiável.

Sobre os pinos de saída, o analógico assume valores entre 0 e 4095, representando a tensão de saída conforme o gás detectado varia. Já o pino digital varia entre 0 e 1, conforme um limiar pré-estabelecido, o qual pode ser ajustado por meio do potenciômetro localizado na parte superior. Enquanto o valor estiver abaixo do limiar, o estado será *LOW*, mas quando a concentração de gás atinge esse limiar, o led interno da própria placa do sensor irá acender e assumir o estado *HIGH*. Para que o limiar seja definido por meio do programa e para haver uma visualização mais efetiva dos níveis do gás no computador, apenas o pino analógico será conectado.

2.8 Sensor de Poeira

Como foi citado anteriormente, a necessidade de realizar o monitoramento e controle de partículas suspensas no ar ou nas superfícies dentro dos ambientes das salas limpas aumentam o risco de contaminação dos produtos a serem trabalhados, o dispositivo se encontra na Figura 10. Para isso, o dispositivo escolhido para realizar essa medição foi fabricado pela *Samyoung S&C Co., Ltd*, o *DSM501a*, e é um sensor de poeira e fumaça capaz de detectar a presença de partículas no ar.

Figura 10 - Visão superior e inferior do sensor DSM501a



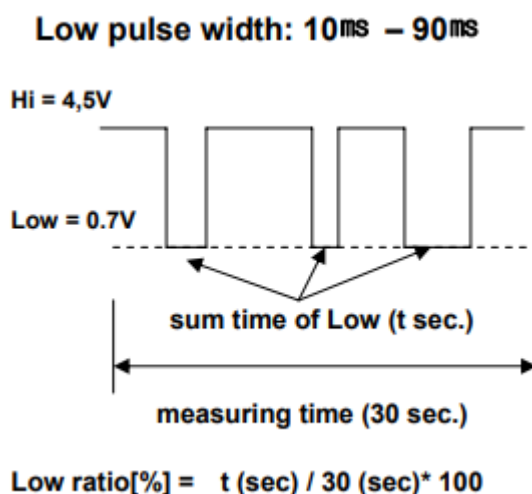
Fonte: De Autoria própria (2024).

O DSM501a precisa ser alimentado por uma tensão de 5 V e, antes de utilizá-lo propriamente, é importante que haja um tempo de pelo menos 30 minutos para garantir a estabilidade do sensor. Além disso, outras orientações chamam a atenção ao consultar o *datasheet* deste dispositivo (2014, p. 11), como não mexer nos dois potenciômetros presentes, evitar de desmontá-lo e não utilizá-lo para situações de emergência ou incêndio. Outra orientação prevista é de limpá-lo e realizar manutenções periódicas, caso seja utilizado em aplicações com muita fumaça e poeira. Isso não se aplica ao caso em questão, pois o ambiente da sala limpa possui um controle rigoroso sobre esses elementos no ambiente.

Este módulo precisa ser instalado na vertical e é projetado para detectar partículas com tamanho maior que um micrômetro. Como por exemplo, poeira doméstica, esporos, pólen, fumaça de cigarro e mofo. As lentes do módulo já são revestidas com polímero anti-estática e anti-poeira, porém, para obter uma performance mais eficiente, necessitam que sejam limpas a cada três meses. Desse modo, o sensor é capaz de realizar o monitoramento das partículas no ar, para garantir que sua concentração no ar mantenha-se controlada na sala limpa, de acordo com as normas da Anvisa, a qual será mostrada na Seção 2.11.

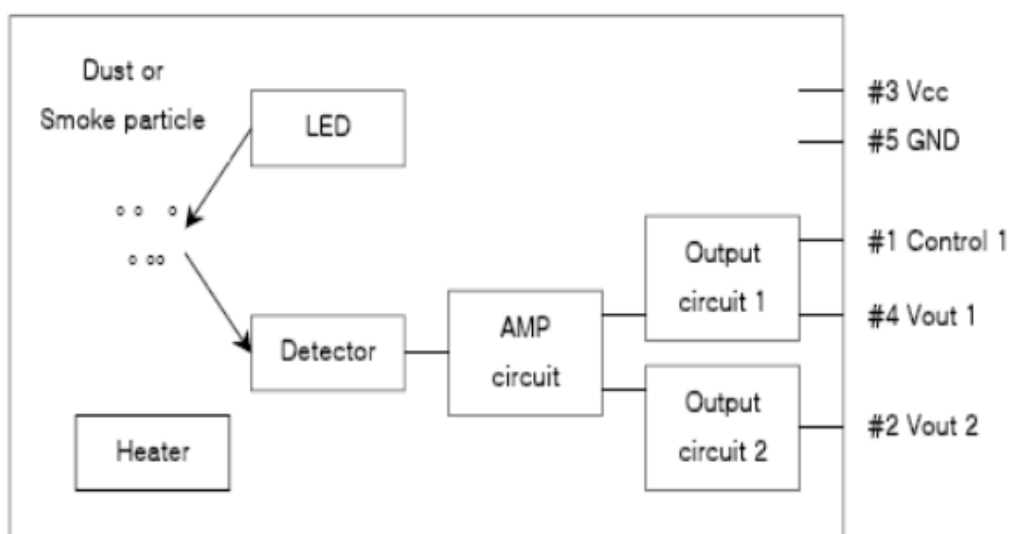
A detecção dessas partículas é feita por meio de um filtro que contabiliza a quantidade de tempo em que há a presença de partículas em relação à quantidade total de ar avaliado. Esse cálculo é feito utilizando a soma do tempo em que as partículas são detectadas sobre o *Low Ratio[%]*, o qual é medido utilizando a soma do tempo em que o sensor está no estado *Low*, ou seja, sem detectar partículas durante um tempo preestabelecido. Sabendo disso, a representação destes pulsos é mostrada na Figura 11.

Figura 11 - Conjunto de pulsos do DSM501a para calcular o *Low Ratio*



Fonte: *Dust Sensor Module P/N: DSM501 Specifications*. 2024.

Figura 12 - Representação em blocos da arquitetura do módulo



Fonte: *Dust Sensor Module P/N: DSM501 Specifications*. 2024.

Primeiramente, da mesma forma que os sensores já citados, há o fio de alimentação *Vcc* e o terra (*GND*), um fio de controle, que não será usado, e os fios de saída $PM_{1,0}$ e $PM_{2,5}$. Cada uma dessas duas saídas tem um alcance diferente em relação às partículas a serem detectadas, pois o $PM_{1,0}$ tem a capacidade de detectar partículas de até $1 \mu m$ e o $PM_{2,5}$ é feito para partículas menores que $2,5 \mu m$. A instalação do módulo deve ser realizada na vertical, com a lente posicionada para

baixo, pois a posição do aquecedor é pensada para que o aumento da temperatura do ar perto da lente realize uma espécie de convecção que atrai o ar para fluir e detectar as partículas, como na Figura 12.

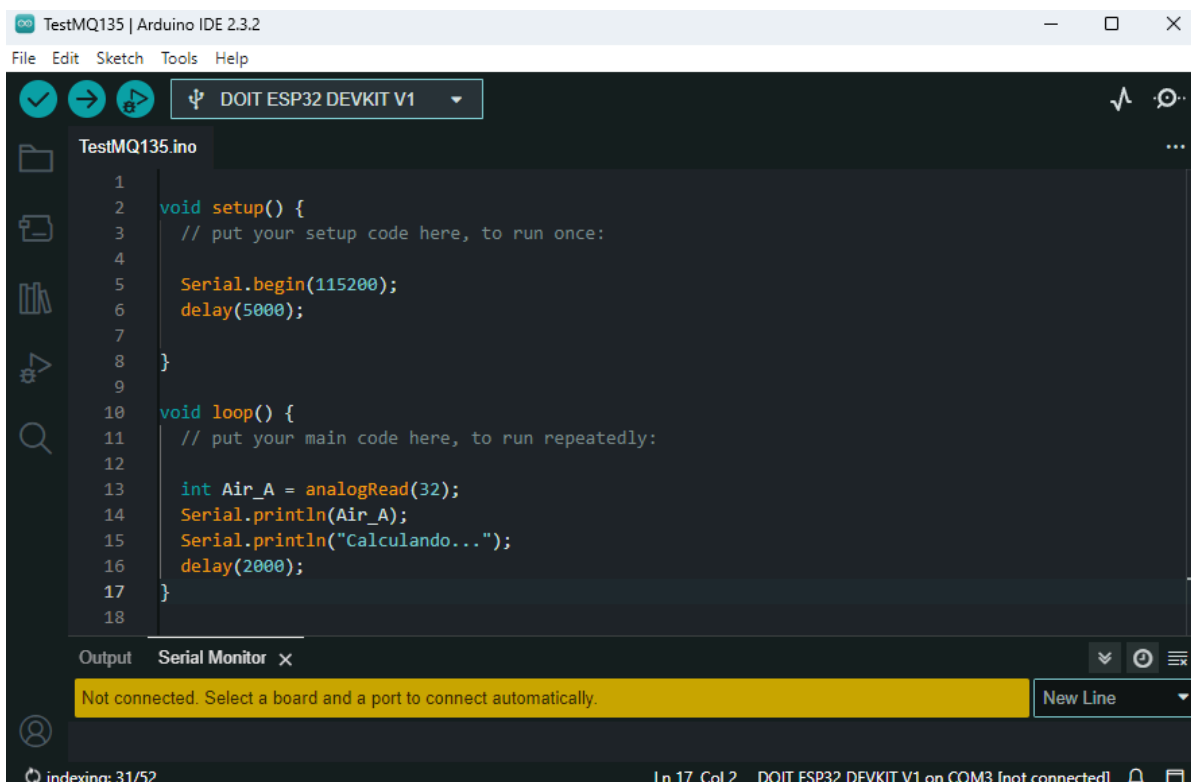
Para uma melhor visualização, o sensor mede a concentração dessas partículas em microgramas por metro cúbico. Porém, as medidas de comparação da qualidade de salas limpas são referenciadas em número de partículas por metro cúbico. Dessa forma, um cálculo de conversão deve ser feito para adaptar os novos valores. Segundo Roy, Bose, Sarddar (2017, p. 1679), para o $PM_{2.5}$, considera-se que a massa dessas partículas é $5,89 \times 10^{-7}$ g, um fator que será utilizado para a conversão durante a análise dos experimentos.

2.9 Arduino IDE

O Arduino IDE é um ambiente de desenvolvimento utilizado para programar, principalmente, a placa Arduino. Porém, esse programa possui compatibilidade com diversos microcontroladores disponíveis no mercado, incluindo diversos da série Esp. Devido a já ter uma maior familiaridade com este ambiente, sabendo melhor sobre suas funcionalidades e interface, será optado pelo uso do Arduino IDE, apesar da empresa do Esp32, a *Espressif*, possuir uma plataforma própria, representada na Figura 13.

O ambiente em questão possui funcionalidades interessantes que facilitam o desenvolvimento dos programas, por exemplo: A ferramenta de pesquisa de bibliotecas ajuda bastante na instalação delas, boa navegação entre os arquivos dentro do programa e consegue trocar entre as placas rapidamente. Além disso, o próprio programa possui uma ferramenta de exibição de variáveis por meio de gráficos, ajudando a visualização de valores flutuantes no tempo.

Figura 13 - Interface do Arduino IDE



Fonte: De autoria própria (2024).

Outro fator para justificar a utilização do Arduino IDE é pela disponibilidade de material de consulta na internet. Pela placa Arduino ser muito utilizada como método de aprendizagem na academia, há muito conteúdo feito nesse ambiente, até quando se utiliza placas com o *Esp8266* e o *Esp32*. Assim, utilizar um programa mais familiar e com mais oportunidades de tirar dúvidas traz mais vantagens ao projeto.

2.10 ThingSpeak

O *ThingSpeak* é uma plataforma *IoT* da empresa *MathWorks*, o qual possui uma versão gratuita, capaz de realizar *uploads* de dados e ter uma visualização destes por meio de gráficos. Dessa forma, a plataforma realiza a função ser um meio de consulta remoto de dados do *Esp32*, o qual é responsável por coletar os dados a serem mandados.

Por se tratar de uma versão gratuita, há certas limitações em relação à sua versão completa. Por exemplo, a limitação de quatro canais cadastrados, com outras palavras, quatro projetos. Além disso, a plataforma só envia mensagens a cada 15

segundos e possui o limite máximo de até 3 milhões de mensagens ao ano. Ainda que existam essas limitações, os recursos disponibilizados são suficientes para o projeto.

O envio de dados por meio do *ThingSpeak* é realizado por meio de uma requisição *HTTP (HyperText Transfer Protocol)* ao servidor da plataforma. trata-se de uma *string* contendo as informações a serem enviadas do *socket TCP client*, o “remetente”, em outras palavras, ao *socket TCP server*, o “destinatário”, que é o próprio servidor do ThingSpeak, por meio da porta 80. Além disso, a plataforma estabelece uma espécie de chave de acesso para canal ser editado por meio de um código que deve ser inserido no programa de comunicação do Esp32, com o objetivo de sincronizar o canal aos dados.

Dentro da plataforma, quando a comunicação é bem sucedida e os dados ficam disponíveis para a visualização dos campos, os quais são os espaços para os gráficos. Há várias opções de customização desses gráficos, como a quantidade máxima de pontos ou de dias, o tipo de gráfico, limites e suas cores. O ThingSpeak possui o limite de 8 campos por canal, isso será mais que suficiente, pois é esperado que só sejam necessários cerca de 4 deles. Tendo isso em vista, os valores a serem enviados são: Temperatura, umidade, pressão e presença de gases no ambiente.

2.11 Salas Limpas

De acordo com a NBR 13413, uma sala limpa pode ser definida como um ambiente controlado, onde sistemas de ar, filtragem e procedimentos visam manter baixos níveis de partículas no ar. Estas salas são essenciais não só em áreas de nutrição e saúde, mas também em diversos processos industriais que exigem alta precisão, como microeletrônica e pintura. Sua principal função é garantir a qualidade e a integridade dos processos através do controle rigoroso em um ambiente limpo de impurezas no ar.

Tendo isso em vista, essas partículas são definidas como diminutas porções de matéria, essenciais para a compreensão e aplicação no desenvolvimento de salas limpas. Porém, não é suficiente apenas contar partículas, é necessário

determinar quais dimensões e tipos podem impactar os processos específicos de cada usuário.

Considerando a aplicação do sistema no contexto de salas limpas para a manipulação de medicamentos e utilizando como base o projeto de salas limpas de Britto (2018), os parâmetros de temperatura, umidade, pressão ambiente e de partículas precisam ser controlados. A pressão diferencial tem que estar entre 5 e 20 pa, com 15 a 30 °C de temperatura e com até 75% de umidade relativa do ar. Já em relação à quantidade de partículas por volume de ar, a tolerância dos tipos de partículas para cada uma das classificações segundo a ISO se encontram na Figura 15, cujas classificações de 1 a 5 podem se encaixar na indústria farmacêutica conforme os componentes trabalhados.

A presença dessas partículas criam condições para a contaminação de componentes com alta sensibilidade a fatores externos e isso pode ocorrer por reações químicas, alterações em propriedades físicas (eletromagnéticas ou mecânicas, por exemplo) e farmacopeicas (toxicidade, interação medicamentosa ou degradação biológica). Operar em ambientes completamente livres de partículas é economicamente inviável, sendo algo possível apenas em condições de vácuo absoluto, porém, há formas encontradas para minimizar a presença de partículas capazes de contaminar estes ambientes controlados.

A geração dessas partículas ocorre por certos meios, como o desprendimento ou liberação de partículas que compõem os materiais constituintes, pinturas, resíduos de lubrificantes, incrustações e entre outros. Outro modo é em que os próprios componentes processados nos ambientes podem sofrer com a atomização (esse desprendimento de partículas) e deixá-las em suspensão no ar ou sendo depositadas nas superfícies expostas.

Um conjunto de salas limpas pode incluir várias salas com diferentes exigências de controle de contaminação, visando proteger ou conter produtos e processos. Para evitar contaminação de ambientes adjacentes menos limpos, as salas limpas devem ter pressão estática maior ou controle da velocidade do ar, fluindo do mais limpo para o menos limpo. Da mesma forma que foi citado anteriormente, é importante que haja um mecanismo que controle a pressão das

salas, por meio da construção com os materiais adequados, visando o processo de enclausuramento, realizando a filtragem adequada da circulação de ar, barreiras físicas e entre outras, seguindo uma configuração semelhante à Figura 14.

Levando em consideração a necessidade de monitoramento de grandezas, como a temperatura, pressão, umidade e certos gases em um ambiente de sala limpa, é preciso desenvolver um circuito com sensores ligados à uma rede capaz de visualizar os valores dos parâmetros presentes neste ambiente controlado. Reforçando sobre o projeto básico de salas limpas, é importante que haja o controle desses fatores e de partículas na superfície e no ar, para evitar efeitos adversos nos processos que ocorrem na sala limpa. E, além disso, o monitoramento desses fatores é importante para garantir a integridade e o funcionamento desses mesmos processos. Tendo em vista essas grandezas, segundo a norma ISO 14644-1, as classificações do nível de qualidade das salas limpas levam em consideração o logaritmo referente à quantidade de partículas no ar por metro cúbico, como na tabela da Figura 15.

Figura 14 - Exemplo de sala limpa vista de fora



Fonte: VL Indústria. 2021. Disponível em:
<https://vlindustria.com.br/a-importancia-das-salas-limpas/>. Acesso em: 08 de julho de 2024.

Figura 15 - Concentração de partículas por m³ segundo a ISO 14644-1

Class	Máximo de partículas/m ³						FED STD 209E equivalência
	>=0.1 µm	>=0.2 µm	>=0.3 µm	>=0.5 µm	>=1 µm	>=5 µm	
ISO 1	10	2					
ISO 2	100	24	10	4			
ISO 3	1,000	237	102	35	8		Class 1
ISO 4	10,000	2,370	1,020	352	83		Class 10
ISO 5	100,000	23,700	10,200	3,520	832	29	Class 100
ISO 6	1,000,000	237,000	102,000	35,200	8,320	293	Class 1,000
ISO 7				352,000	83,200	2,930	Class 10,000
ISO 8				3,520,000	832,000	29,300	Class 100,000
ISO 9				35,200,000	8,320,000	293,000	Room Air

Fonte: CMS Científica. 2019. Disponível em:

<https://cmscientifica.com.br/o-que-e-uma-sala-limpa-classificacao-de-ambientes-controlados/#:~:text=Salas%20limpas%20s%C3%A3o%20classificadas%20de,por%20p%C3%A9%20c%C3%ABico%20de%20ar>. Acesso em: 31 de julho de 2024.

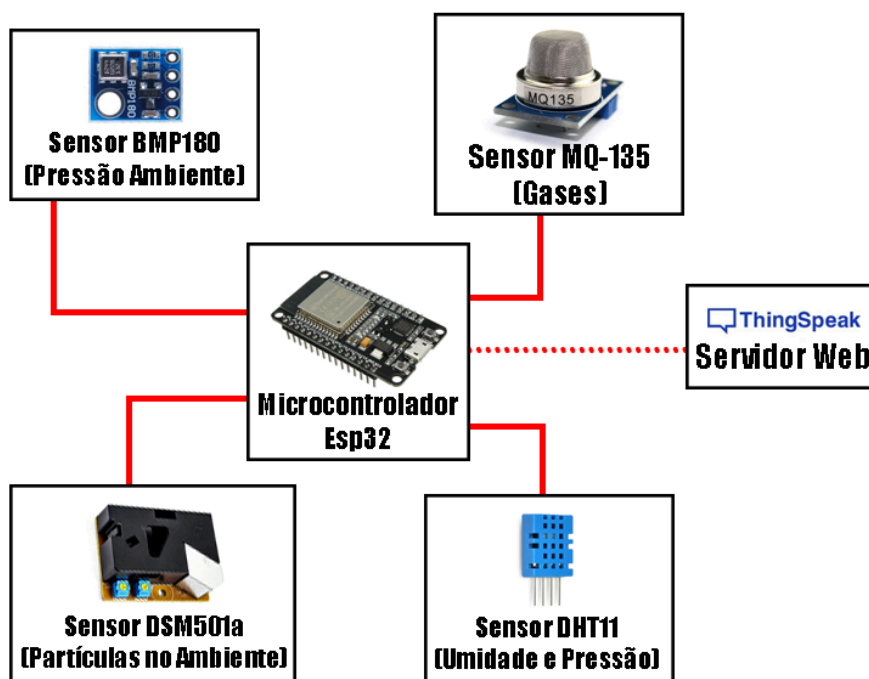
3 METODOLOGIA

3.1 Arquitetura do Projeto

Com o objetivo de realizar as devidas medições de temperatura, umidade, pressão, gases e partículas no ar e que exiba os valores importantes, é necessário planejar uma configuração que permita a coleta de dados por meio dos sensores, o processamento e envio de dados por meio do microcontrolador até o servidor Web. De acordo com a Figura 16, o sistema deve estar em um ambiente ligado à internet por meio do roteador Wi-Fi enviando os dados que são coletados no ambiente pelos devidos sensores.

Devido à impossibilidade de observar a operação do sistema em um ambiente controlado de sala limpa, o circuito será testado em um ambiente doméstico fechado, analisando os valores dos parâmetros já citados neste local. Apesar disso, ao verificar o funcionamento dos testes no projeto neste ambiente e entendendo o alcance das medições físicas dos próprios sensores estando dentro dos padrões estabelecidos pelas normas, seria possível validar a funcionalidade do sistema proposto, considerando as margens de erro e as limitações de teste.

Figura 16 - Representação da arquitetura do projeto



Fonte: De Autoria própria (2024).

Além disso, para atestar o funcionamento dos sensores e realizar as devidas calibrações antes da integração em conjuntos desses com o microcontrolador, um teste individual foi realizado com os quatro sensores principais. Após o teste individual dos sensores, o circuito completo será montado na protoboard ligado ao *Esp32* devidamente e usando a ferramenta Arduino IDE para programar o microcontrolador e estabelecer a conexão com a plataforma *ThingSpeak*, por meio do roteador *Wi-Fi* do ambiente. Assim, com os sensores integrados ao sistema completo, o sistema ficará cerca de 48 horas ligado no ambiente escolhido para o recolhimento de dados e comparar com os valores reais, com o objetivo de avaliar o desempenho do sistema como um todo.

Os materiais utilizados foram:

- Placa *Esp32 Doit Devkit V1*;
- Sensor de partículas e fumaça *Samyoung DSM501a*;
- Sensor de qualidade do ar Hanwei *MQ-135*;
- Sensor de pressão e temperatura *Bosch BMP180*;
- Sensor de temperatura e umidade *DHT11*;
- Protoboard, jumpers e resistores;
- Computador *desktop*.

3.2 Testes Individuais

Para realizar os testes, com cada um dos sensores, foi preciso pensar na configuração de montagem do circuito e programação do código a ser enviado para o microcontrolador. Já que a função desta etapa é avaliar o desempenho da medição e preparar o código de cada sensor antes da integração em conjunta, os devidos valores medidos foram exibidos por meio do monitor serial do Arduino IDE.

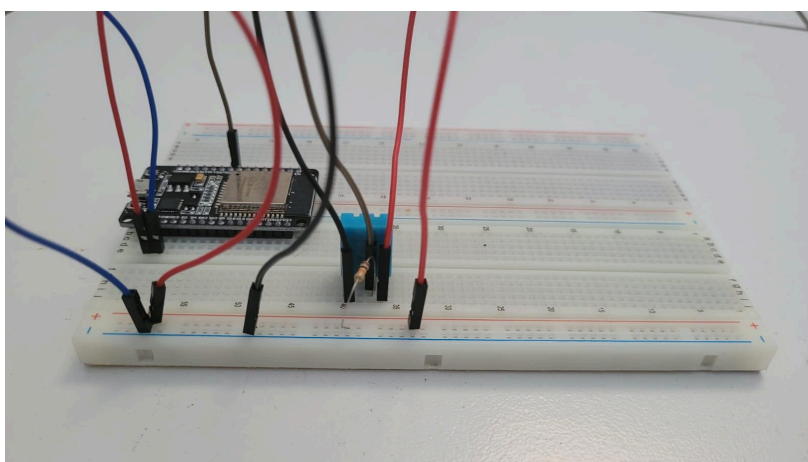
Cada sensor foi ligado ao *Esp32* separadamente, de acordo com as orientações do fabricante, e o código foi programado utilizando o Arduino IDE, utilizando a linguagem Arduino, a qual é quase inteiramente baseada em C++ com algumas alterações, e bibliotecas próprias recomendadas para alguns dos sensores. Um ponto a ser evidenciado é que certos sensores necessitam de tempos diferentes de pré aquecimento, antes do uso, e essas informações mais detalhadas se encontrarão nos tópicos seguintes.

3.2.1. Sensor de Temperatura e Umidade

A montagem do circuito para o sensor *DHT11* foi feita seguindo as orientações do *datasheet* do fabricante e pensando no código programado no *IDE*, como na Figura 17. Como o sensor necessitava de 3,3 V, o pino *Vcc* deste foi conectado a uma trilha da protoboard ligada à porta do *Esp32* com esta tensão de saída. Além disso, o pino de *GND* do sensor foi ligado à trilha com um *jumper* conectado à porta *GND* do microcontrolador.

Após estabelecer a alimentação do sensor, foi necessário definir qual porta do *Esp32* serviria como a leitura para os dados de temperatura e umidade, já que o sensor envia esses valores por meio de um único pino. Deste modo, a porta escolhida foi a D26, pois esta possui um circuito de conversão de dados analógicos para digitais, o que é importante para a leitura das informações do sensor, como foi anteriormente explicado.

Figura 17 - Circuito montado do DHT11



Fonte: De Autoria própria (2024).

Figura 18 - Código referente ao sensor DHT11

```

1  #include <DHT.h>
2
3  DHT dht(26, DHT11); // Define o pino D26 como leitura
4
5  void setup(void) {
6      Serial.begin(115200);
7      dht.begin();
8  }
9
10 void loop(void) {
11     Temperatura(); // Função de leitura de temperatura
12     Umidade(); // Função de leitura de umidade
13     delay(2000);
14 }
15

```

Fonte: Autoria própria (2024).

Como pode ser observado na Figura 18, a primeira linha é usada para incluir a biblioteca do sensor *DHT11*, a qual possui algumas funções prontas para leitura de temperatura e umidade. De acordo com a Figura 19, as funções *Temperatura()* e *Umidade()* foram criadas e usam as funções da biblioteca *readTemperature()* e *readHumidity()* para ler os valores do sensor de temperatura e umidade, respectivamente. Após isso, há uma condicional para verificar se os valores armazenados na variável são números ou não. Caso não seja um número, a função retornará 0 e a mensagem “Falha na Leitura” aparecerá no monitor serial.

Figura 19 - Código das funções *Temperatura()* e *Umidade()*

<pre> float Temperatura() { float t = dht.readTemperature(); if (isnan(t)) { Serial.println("Falha na Leitura"); return 0; } else { Serial.print("Temperatura atual = ") Serial.print(t); Serial.println(" *C") return t; } } </pre>	<pre> float Umidade() { float h = dht.readHumidity(); if (isnan(h)) { Serial.println("Falha na Leitura"); return 0; } else { Serial.print("Umidade Atual = ") Serial.print(h); Serial.println("%") return h; } } </pre>
--	---

Fonte: De Autoria própria (2024).

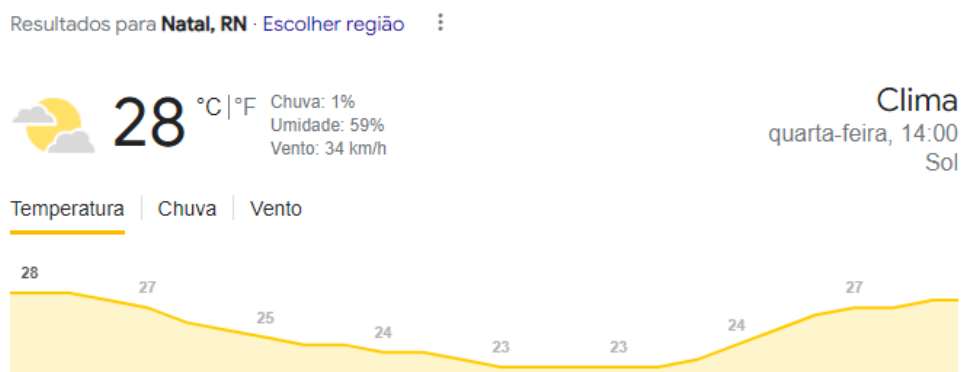
Para aumentar a preservação do sensor e do microcontrolador, um resistor de 10 k Ω foi inserido entre o pino de dados do sensor e a trilha do Vcc do *Esp32*. Com o circuito pronto e o programa devidamente enviado para o microcontrolador, cerca de uma hora foi esperada com o circuito ligado para observar os dados de uma forma mais estável. Assim, a temperatura registrada foi de 28,70 °C e a umidade foi de 61,00% e os valores foram comparados com o que foi visto no site *weather.com*. Esses resultados se encontram na Figura 20 e a comparação com os valores do site está na Figura 21.

Figura 20 - Leitura do sensor de temperatura e umidade após uma hora

```
14:22:02.375 -> Temperatura atual = 28.70 *C
14:22:02.375 -> Umidade Atual = 61.00%
14:22:02.411 ->
14:22:03.427 ->
14:22:03.427 -> Temperatura atual = 28.70 *C
14:22:03.427 -> Umidade Atual = 61.00%
14:22:03.427 ->
14:22:04.489 ->
14:22:04.489 -> Temperatura atual = 28.70 *C
14:22:04.489 -> Umidade Atual = 61.00%
14:22:04.489 ->
14:22:05.512 ->
14:22:05.512 -> Temperatura atual = 28.70 *C
14:22:05.512 -> Umidade Atual = 61.00%
```

Fonte: De Autoria própria (2024).

Figura 21 - Condições meteorológicas segundo o *weather.com* no momento da medição.



Fonte: Weather. 2024. Disponível em:

<https://weather.com/weather/today//19fd33420ee4c8b43e31f59094c09611edf1a68c9791aadab1fdc5428fbfe933>. Acesso em: 17 de julho de 2024.

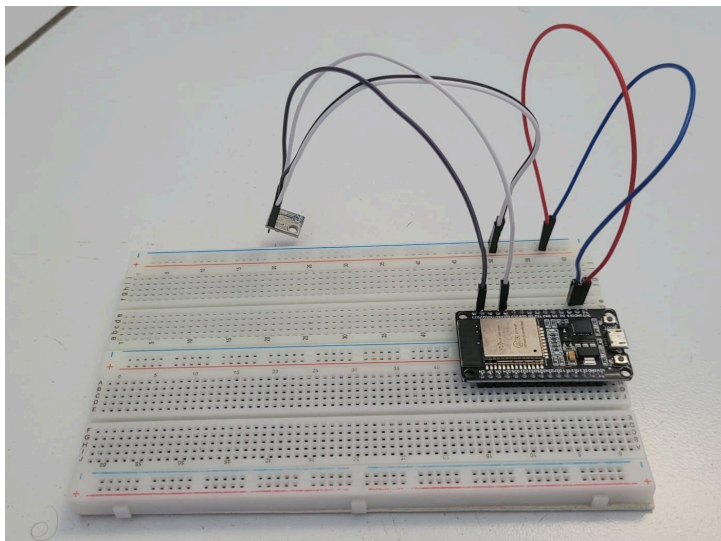
3.2.2. Sensor de Pressão

Para o teste do sensor de pressão, também foi necessário conectar o módulo *BMP180* ao microcontrolador por meio da protoboard. Este sensor tem uma particularidade, ele possui pontos de conexão fêmea e os pinos de conexão machos vieram separadamente. O ideal nessa situação seria soldá-los, para que a conexão fique mais firme, porém, as conexões são tão pequenas e próximas umas das outras, que seria muito difícil e arriscado realizar essa solda.

Ou seja, a solda podia ser mal posicionada e juntar entre as conexões ou correr o risco de estragar o sensor de outra forma. Devido a essa situação, testes com diferentes configurações foram feitos. Duas configurações foram experimentadas: A primeira foi encaixando os pinos machos avulsos nas entradas fêmeas, de um lado, e conectando a outra parte à protoboard, a qual é ligada ao microcontrolador por meio das trilhas com o microcontrolador, de acordo com a Figura 22. E a outra forma foi ligar as entradas fêmeas do *BMP180* diretamente com *jumper*s, sem solda.

Diferentemente do programa do *DHT11*, a definição dos pinos é mais obscura no código, porém, já que o sensor necessita de conexão especificamente com o *SDA* e o *SCL* do *Esp32*, as portas D22 e D21 foram conectadas, respectivamente. Tendo isso em vista, o programa, por padrão, utiliza essas portas para conseguir a leitura dos dados da pressão atmosférica, representado na Figura 23. Caso fosse necessário alterar as portas, poderia-se incluir a biblioteca *Wire* e usar a função para definir as novas portas.

Figura 22 - Circuito montado com o BMP180



Fonte: De Autoria própria (2024).

Figura 23 - Código de inicialização do sensor BMP180

```
1  #include <Adafruit_BMP085.h>
2
3  Adafruit_BMP085 bmp;
4
5  void setup() {
6      Serial.begin(115200);
7      if (!bmp.begin()) {
8          Serial.println("Sensor inválido");
9          while (1) {}
10     }
11     int Press = 0;
12 }
```

Fonte: De Autoria própria (2024).

Como foi citado na fundamentação teórica, a biblioteca utilizada foi referente ao sensor BMP085, porém, também é compatível com o BMP180. Após isso, o programa faz uma verificação da conexão com o sensor, caso não seja bem sucedida, a mensagem “Sensor inválido” seria exibida no monitor serial. A variável *Press* é inicializada para armazenar os valores da pressão atmosférica lidas pelo sensor e o objeto *bmp* é criado com as propriedades da classe *Adafruit_BMP085* para usar as funções referentes ao sensor de pressão, como na Figura 24.

Figura 24 - Código de leitura e exibição da pressão ambiente

```

14 void loop() {
15
16     Press = bmp.readPressure();
17     Press = Press*(0.01);
18
19     Serial.print("Pressão Ambiente atual: ");
20     Serial.println(Press);
21     Serial.println(" hPa");
22
23     delay(1000);
24 }

```

Fonte: De Autoria própria (2024).

A função *readPressure()* lê o valor de tensão da porta de dados e realiza a conversão de valores para Pascal. Normalmente, a medida padrão utilizada para a pressão atmosférica é hPa (hectopascal). Logo, foi feita a conversão multiplicando os valores encontrados por 0,01 para a exibição dos valores no monitor serial ser mais compatível com os valores da condição meteorológica no momento de medição.

Figura 25 - Medição da pressão pelo sensor BMP180

```

23:39:48.555 -> Pressao Ambiente atual: 1016.71 hPa
23:39:49.587 -> Pressao Ambiente atual: 1016.75 hPa
23:39:50.651 -> Pressao Ambiente atual: 1016.68 hPa
23:39:51.692 -> Pressao Ambiente atual: 1016.70 hPa
23:39:52.749 -> Pressao Ambiente atual: 1016.67 hPa
23:39:53.756 -> Pressao Ambiente atual: 1016.68 hPa
23:39:54.813 -> Pressao Ambiente atual: 1016.61 hPa
23:39:55.871 -> Pressao Ambiente atual: 1016.63 hPa
23:39:56.935 -> Pressao Ambiente atual: 1016.65 hPa
23:39:57.978 -> Pressao Ambiente atual: 1016.67 hPa

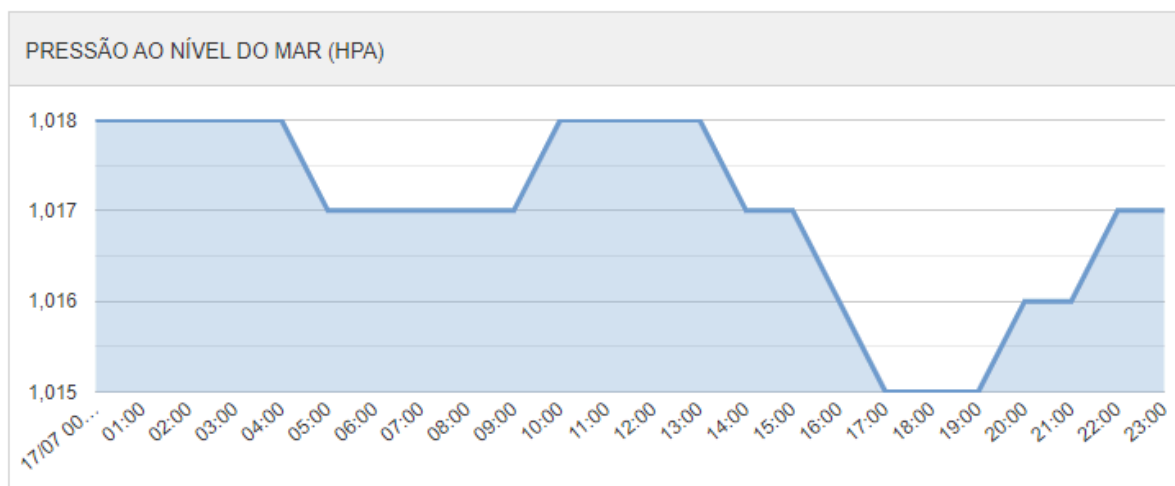
```

Fonte: De Autoria própria (2024).

De acordo com a Figura 25 e pelo fato das leituras estarem com uma pequena variação mesmo após um tempo de uma hora com o sensor funcionando para estabilizar as medições, os últimos 10 valores foram registrados e fez-se uma média para compará-los com os valores reais de pressão atmosférica segundo o site *tempo.com*. Ao calcular a média dos últimos 10 valores medidos pelo BMP180, o

valor medido considerado é 1016,68 hPa. Ao comparar com a curva do site, o valor atribuído entre 23:00 e 00:00 equivale a 1017 hPa, satisfatoriamente próximo do que foi medido, representado na Figura 26.

Figura 26 - Curva da pressão atmosférica ao longo do dia 17/07 em Natal-RN



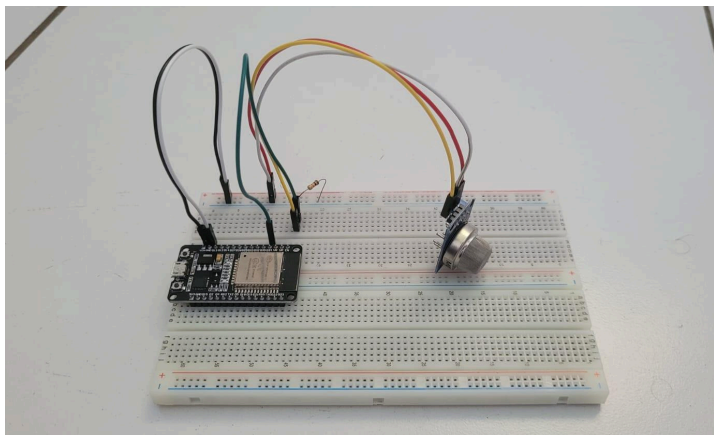
Fonte: Tempo. 2024. Disponível em: <https://www.tempo.com/natal.htm>. Acesso em: 17 de julho de 2024.

3.2.3. Sensor de Qualidade do Ar

Para realizar o teste com o sensor MQ-135, foi necessário tomar certos cuidados em relação à tensão de alimentação do sensor. Pois, diferentemente dos sensores BMP180 e DHT11, a tensão recomendada é de 5 V. Essa situação pode ser perigosa para o Esp32, devido a suas portas de entrada suportarem até cerca de 3,3 V. Para contornar esse empecilho, foi optado por ligar um resistor entre o pino de dados e o GND. O circuito foi montado conforme a Figura 27.

A porta D34 foi escolhida para ler os dados analógicos de saída do sensor, pois esta também possui o conversor analógico-digital. O pino digital do MQ-135 foi deixado sem conexão neste teste, pois o objetivo principal foi observar as leituras analógicas do sensor no ambiente escolhido. Um resistor de 10 kΩ foi conectado por segurança, para evitar que a tensão de saída chegue a 5 V.

Figura 27 - Circuito montado do MQ-135



Fonte: De Autoria própria (2024).

A programação foi feita de forma mais simples do que a dos sensores anteriores, pois não foi preciso de bibliotecas específicas. Dessa forma, o código apenas define o pino D34 para receber os dados e exibe os dados no monitor serial diretamente. Como a unidade de medida é referente apenas ao nível de tensão, o comportamento do sensor foi observado ao longo do tempo no ambiente escolhido para entender os níveis normais. A programação feita conforme as Figuras 28 e 29.

Figura 28 - Código de inicialização do sensor *MQ-135*

```
1  #define Pin_MQ135 34
2
3  int valor_MQ135 = 0;
4
5  void setup(){
6
7    Serial.begin(115200);
8
9  }
10
```

Fonte: De Autoria própria (2024).

Figura 29 - Código da medição do sensor MQ-135

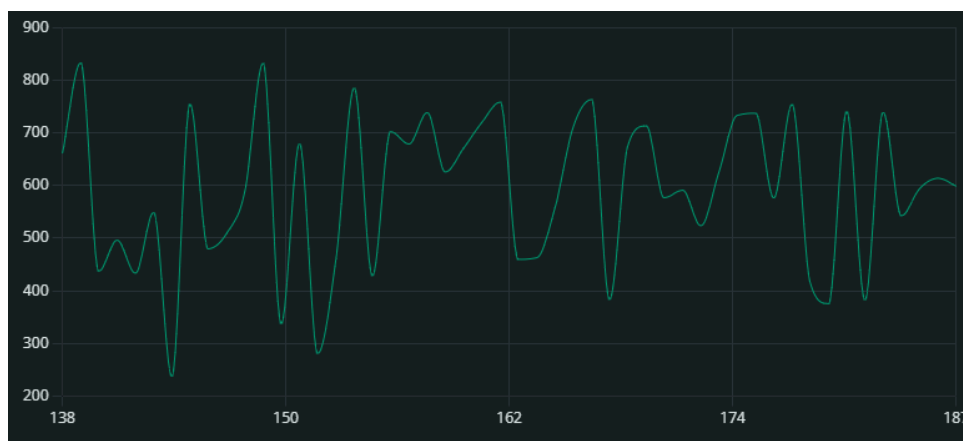
```
10 void loop(){  
11  
12 valor_MQ135 = analogRead(Pin_MQ135);  
13 Serial.print("AirQua = ");  
14 Serial.println(valor_MQ135);  
15 delay(5000);  
16  
17 }
```

Fonte: De Autoria própria (2024).

Com o código enviado para o *Esp32* e o circuito montado com o sensor, o circuito foi deixado para o preaquecimento. Pois, como já foi citado, os sensores da série *MQ*, geralmente, necessitam de um tempo maior de preaquecimento para suas leituras serem mais efetivas. Dessa forma, o sensor foi deixado funcionando por cerca de 24 horas, para que seu comportamento seja avaliado após esse período.

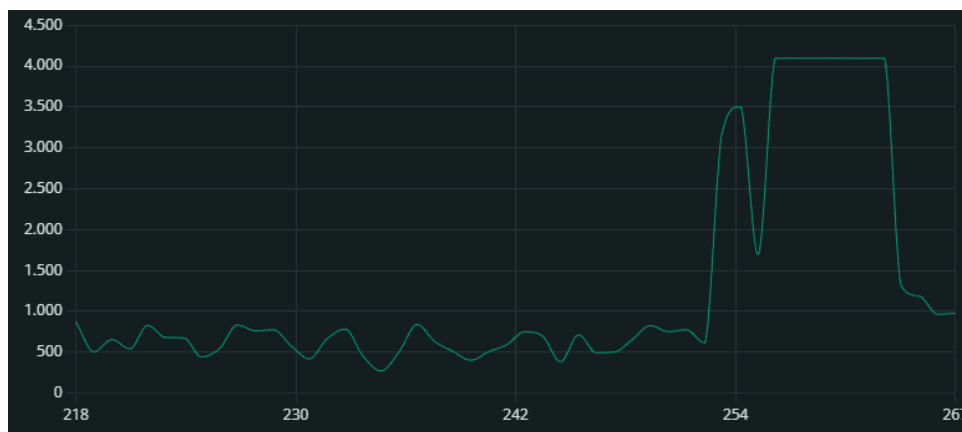
Após as 24 horas, os valores do sensor foram observados por meio da ferramenta de plotagem de gráfico de variáveis do Arduino IDE. É possível perceber que os números variam entre, cerca de, 220 e 830. À primeira vista, o sensor pareceu instável, porém, ao expor o *MQ-135* a álcool em gel, as partículas reagiram bem com o sensor e elas foram detectadas com discrepância, chegando a alcançar o valor máximo do sensor (4095), conforme a Figura 30 e a Figura 31.

Figura 30 - Curva dos valores medidos pelo sensor MQ-135 em condições ambientes normais.



Fonte: De Autoria própria (2024).

Figura 31 - Curva de valores medidos pelo sensor MQ-135 à exposição de álcool



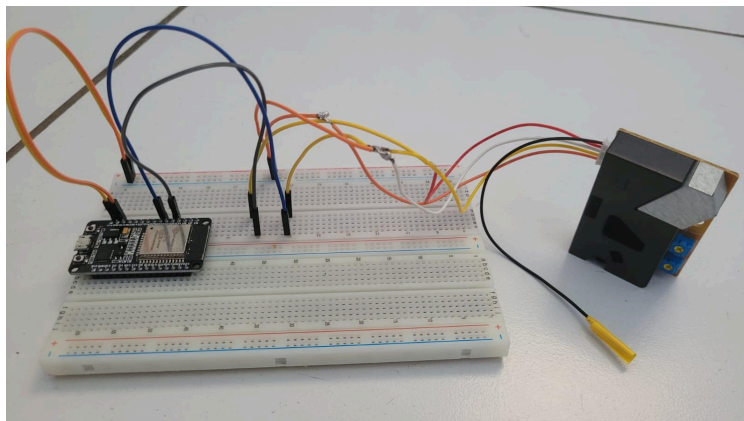
Fonte: De Autoria própria (2024).

3.2.4. Sensor de Poeira

O sensor DSM501a tem uma configuração um pouco mais sofisticada em comparação aos outros sensores, principalmente na questão do código. Da mesma forma que o MQ-135, esse sensor necessita da alimentação de 5V no Vcc e é necessário tomar cuidado para não sobrecarregar as portas de 3,3 V do Esp32. Pensando nisso, a mesma solução foi pensada utilizando resistores de 1,5 k Ω para limitar a tensão de saída dos pinos de dados $PM_{1,0}$ e $PM_{2,5}$ que iriam diretamente para as portas D34 e D35 do microcontrolador, respectivamente.

As conexões de metal dos fios que são acoplados no DSM501a eram muito curtas e não chegavam a entrar fundo o suficiente na protoboard. Devido a isso, foi necessário descascar mais a ponta dos fios para criar uma abertura maior de conexão, porém, deixando-as menos firmes e mais frágeis. O fio de controle foi deixado sem conexão, como na Figura 32.

Figura 32 - Circuito montado do DSM501a



Fonte: De Autoria própria (2024).

Em relação ao código, foi difícil encontrar materiais com orientações claras da programação e do funcionamento do sensor. Como foi citado anteriormente, as partículas entram pela cavidade do sensor, são detectadas e isso interfere no pulso do sensor, que muda sua forma conforme há a presença das partículas. As duas saídas funcionam dessa forma, tanto a $PM_{1,0}$ e $PM_{2,5}$, e o resto da contabilização precisa ser pensada por meio do programa. O código foi feito utilizando, como referência, o site elecrow.com, que programou o sensor pensando em uma aplicação com *Esp8266*. Dessa forma, o código foi adaptado para o *Esp32* e para atender às necessidades da aplicação em questão.

Figura 33 - Código do sensor *DSM501a* para definição das variáveis

```

1  #include<string.h>
2  #define PM1PIN 34
3  #define PM25PIN 35
4  byte buff[2];
5  unsigned long durationPM1;
6  unsigned long durationPM25;
7  unsigned long starttime;
8  unsigned long endtime;
9  unsigned long sampletime_ms = 30000;
10 unsigned long lowpulseoccupancyPM1 = 0;
11 unsigned long lowpulseoccupancyPM25 = 0;

```

Fonte: De Autoria própria (2024).

O código da Figura 33 se baseia no estabelecimento das variáveis para o pulso que calculam a presença das partículas no ar, utilizando o tempo de amostragem de 30 segundos. Dessa forma, o programa vai utilizar o tempo de início e de término do pulso, comparando com o tempo de amostragem, para que a análise ocorra sempre entre intervalos de 30 segundos. Após isso, o programa pega esse tempo de amostragem e compara com o tempo em que o pulso está no estado *low*, que ocorre inversamente proporcional à medida que as partículas passam pela cavidade.

Foi criada a função *calculateConcentration()* na Figura 34 que utiliza esse tempo como base e calcula a razão entre o tempo de ocupação no *low* e o tempo da amostragem. Tendo o valor dessa razão, o programa calcula a concentração de partículas por metro cúbico no ar por meio da equação da curva de tensão de saída do sensor. O valor da concentração é dado em $\mu\text{g}/\text{m}^3$, para se ter uma melhor visualização, como mostrado na Figura 35.

Figura 34 - Código do sensor DSM501a para cálculo da concentração

```

39 void loop()
40 {
41     durationPM1 = pulseIn(PM1PIN, LOW);
42     durationPM25 = pulseIn(PM25PIN, LOW);
43     lowpulseoccupancyPM1 += durationPM1;
44     lowpulseoccupancyPM25 += durationPM25;
45
46     endtime = millis();
47     if ((endtime-starttime) > samptime_ms)
48     {
49         float conPM1 = calculateConcentration(lowpulseoccupancyPM1,30);
50         float conPM25 = calculateConcentration(lowpulseoccupancyPM25,30);
51
52         Serial.print("PM1 ");
53         Serial.print(conPM1);
54         Serial.print(" ug/m3");
55         Serial.print(" PM25 ");
56         Serial.print(conPM25);
57         Serial.println(" ug/m3");
58         lowpulseoccupancyPM1 = 0;
59         lowpulseoccupancyPM25 = 0;
60         starttime = millis();
61     }
62 }

```

O sensor ficou cerca de uma hora funcionando para dar tempo de aquecê-lo. Os valores foram sendo acompanhados por meio do monitor serial e exibiram valores com certa variação na medida, por se tratar de um ambiente com circulação de vento.

Figura 35 - Medidas do DSM501a no monitor serial

```
15:12:38.396 -> PM1 46.14 ug/m3 PM25 10.65 ug/m3
15:13:08.452 -> PM1 22.31 ug/m3 PM25 36.38 ug/m3
15:13:38.892 -> PM1 39.70 ug/m3 PM25 36.70 ug/m3
15:14:09.195 -> PM1 48.78 ug/m3 PM25 43.79 ug/m3
15:14:39.461 -> PM1 34.54 ug/m3 PM25 43.04 ug/m3
```

Fonte: De Autoria própria (2024).

3.3 Teste em Conjunto

Com o objetivo de realizar as medições dos parâmetros do ambiente e analisar o desempenho do projeto dos sensores em conjunto de forma simultânea, a montagem foi pensada seguindo a arquitetura citada na Seção 3.1. Dessa forma, foi necessário integrar os sensores ao microcontrolador, tendo em mente os diferentes valores de alimentação, comportar as saídas e entradas de dados conforme as características de cada uma delas e utilizar o espaço da protoboard de forma que acomode todos os circuitos de forma segura.

Além da parte do circuito físico, deve-se também abordar sobre a questão da integração à plataforma *ThingSpeak*, onde foi possível avaliar os resultados obtidos de forma prática e visual. Um ponto a ser destacado é a limitação da versão de teste do *ThingSpeak*, a qual foi suficientemente satisfatória para os testes propostos. Porém, não seria tão eficiente em aplicações mais robustas e a longo prazo, devido ao limite de envio de dados por ano. Apesar dos transtornos, os testes foram adaptados para contornar as limitações da plataforma, de forma que não haja perdas significativas na validação destes.

3.3.1. Configuração da Montagem

A realização da montagem do sistema como um todo foi feita por meio dos testes e experimentações individuais com cada um dos sensores, pensando em como integrá-los de forma que não haja interferências entre eles, ilustrado pela

Figura 36. Exemplificando, os sensores *DHT11* e *BMP180* precisam de alimentação de 3,3 V, o que pode ser fornecido sem problemas devido ao barramento com este nível de tensão fornecido por uma das portas de saída do *Esp32* e que pode ser lido pelas portas de entrada. Por outro lado, os sensores *MQ-135* e *DSM501a* precisam de 5 V, necessitando de um barramento separado ligado ao V_{in} do microcontrolador, o qual não pode ser lido pelas portas do *Esp32* sem cuidados extras.

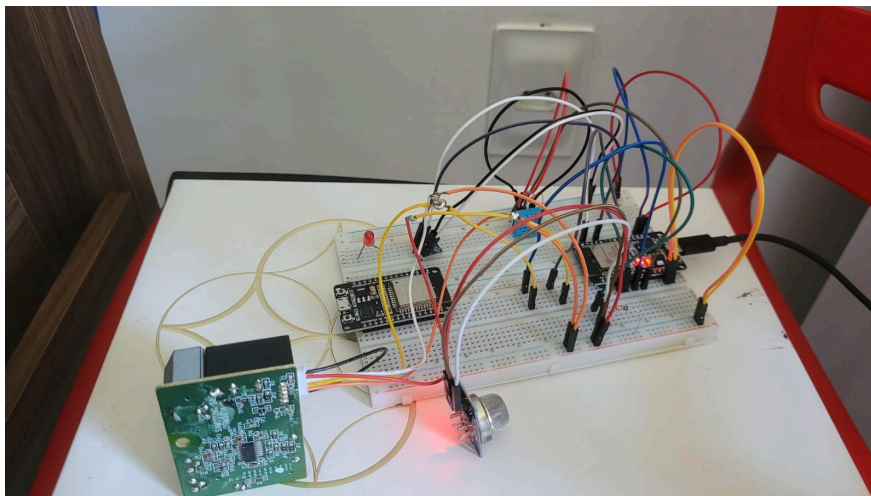
Sabendo disso, a saída dos sensores *DHT11* e *MQ-135* tiveram um resistor de 10 k Ω conectado, em cada, para garantir uma segurança maior. Em relação ao *DSM501a*, optou-se por ligar diretamente às portas do *Esp32*, pois espera-se que a tensão de saída seja baixa neste tipo de aplicação, pois o número de partículas deve ser baixo, na ordem de microgramas por metro cúbico. Ter uma boa noção e organização dos circuitos individuais facilitou a integração deles com o sistema completo. As demais conexões ligadas ao *Esp32* estão situadas na Tabela 1:

Tabela 1 - Portas de conexão dos sensores com *Esp32*.

Sensores	Portas do Esp
Temperatura e Umidade	D26
Pressão Ambiente (<i>SDA</i>)	D21
Pressão Ambiente (<i>SCL</i>)	D22
Presença de Gás	D34
Partículas 1 μm	D35
Partículas 2.5 μm	D33

Um ponto a ser comentado sobre os sensores individualmente é que os fios conectados ao sensor *DSM501a* são bem finos e possuem uma pequena área de contato metálico em suas pontas, dificultando uma conexão estável na protoboard. Como foi citado na Seção 3.2.4., os fios foram desencapados para melhorar a conexão, porém, em consequência disso, os pontos de contato foram perdendo sua forma e sua rigidez, tornando as ligações mais difíceis, conforme iam acontecendo, durante os testes. Para contornar essa situação, *jumpers* foram soldados ligados aos quatro fios utilizados nas portas do sensor *DSM501a*, facilitando a organização.

Figura 36 - Circuito inteiro montado no ambiente doméstico

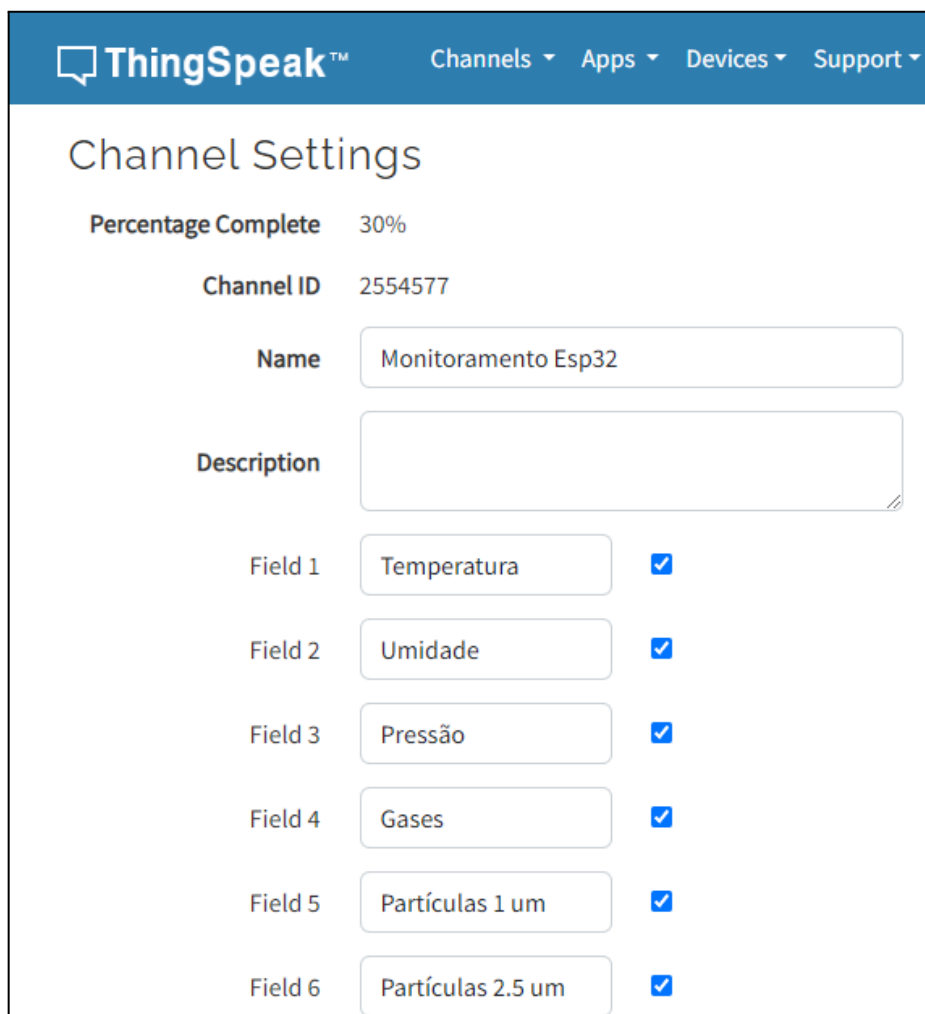


Fonte: De Autoria própria (2024).

3.3.2. Integração ao Thingspeak

Tendo o circuito montado em conjunto com o *Esp32*, foi necessário programar a conexão entre o microcontrolador e a plataforma *ThingSpeak*, utilizando o roteador como comunicador. Para isso, utilizou-se a biblioteca *Thingspeak* da *Mathworks*, que facilitou o estabelecimento da conexão entre o microcontrolador e o canal criado na plataforma *online*. O objetivo de usar a plataforma é conseguir exibir os valores medidos pelos sensores de uma forma mais prática, por meio de um canal composto por campos que exibem os gráficos das medições.

Para isso, foi criado o canal “Monitoramento *Esp32*”, o qual possuía seis canais dedicados para representar os valores dos sensores, que se encontram na Figura 37. Os campos mostravam as medidas de temperatura, umidade, pressão atmosférica, gases, partículas de 1 μm e partículas de 2,5 μm , respectivamente. O *ThingSpeak* gerou a chave *API* para ler e escrever os dados na plataforma, porém apenas a chave de escrita foi utilizada, pois, o objetivo da aplicação utilizou a plataforma para consultar os dados.

Figura 37 - Configurações do canal no *ThingSpeak*

The screenshot displays the 'Channel Settings' interface on the ThingSpeak platform. At the top, there is a navigation bar with the ThingSpeak logo and links for Channels, Apps, Devices, and Support. The main heading is 'Channel Settings'. Below this, the 'Percentage Complete' is indicated as 30%. The 'Channel ID' is 2554577. The 'Name' field contains 'Monitoramento Esp32'. The 'Description' field is empty. Below the description, there are six 'Field' entries, each with a text input and a checkbox:

Field	Value	Checked
Field 1	Temperatura	☒
Field 2	Umidade	☒
Field 3	Pressão	☒
Field 4	Gases	☒
Field 5	Partículas 1 um	☒
Field 6	Partículas 2.5 um	☒

Fonte: De Autoria própria (2024).

Com o propósito de estabelecer a comunicação entre o roteador de internet e o microcontrolador, utilizou-se a biblioteca Wifi, a qual faz a conexão por meio do nome da rede e senha do roteador. Como anteriormente citado, há a limitação do ThingSpeak de 15 segundos entre os envios de dados para a plataforma, que não foi um impeditivo para os testes. Porém, durante as experimentações individuais, foi percebido que os gráficos dos campos nos canais do ThingSpeak possuem um limite de 60 pontos.

Ou seja, ao bater o limite de 60 pontos, o valor mais antigo presente no gráfico é substituído pelo novo. Considerando que o teste a ser feito pelo sistema contendo todos os sensores propõe o tempo de 24 horas de medição, foi preciso calcular o intervalo de tempo entre os envios de dados para a plataforma, de modo

que os gráficos ocupem o limite máximo de pontos. Considerando que o dia possui 86400 segundos e o limite máximo de pontos é de 60, então o intervalo de tempo precisa ser de 1440 segundos, ou seja, 24 minutos, isso pode ser visto na Figura 38. Tendo isso em vista, esse é o valor de atualização dos dados medidos no canal do ThingSpeak.

Figura 38 - Código da configuração do canal

```
23  int intervalSensor = 2000;
24  long prevMillisThingSpeak = 0;
25  int intervalThingSpeak = 1440000; // (24 minutos)
26
27  #include <WiFi.h>
28  const char ssid[] = "Nome_da_Rede";
29  const char password[] = "Senha_da_Rede";
30  WiFiClient client;
31
32  #include <ThingSpeak.h>
33  const long CHANNEL = "Código_do_Canal";
34  const char *WRITE_API = "Chave_API_Escrita";
```

Fonte: De Autoria própria (2024).

Após estabelecer a conexão e o intervalo de tempo das mensagens, o código compara o tempo entre os envios dos dados para atualizar os valores na plataforma ThingSpeak. A função *ThingSpeak.setField()* da biblioteca é usada para definir quais campos seriam ocupados para cada um dos valores medidos pelos sensores para a exibição no canal, como visto na Figura 39. Dessa forma, os valores dos seis campos já descritos foram atualizados com as medições a cada 24 minutos, caso a comunicação estivesse bem estabelecida. O código inteiro, incluindo a parte dos sensores e da comunicação com o ThingSpeak ficará na Seção de Apêndice.

Figura 39 - Código de atualização dos valores no ThingSpeak

```

181  if (millis() - prevMillisThingSpeak > intervalThingSpeak) {
182
183      ThingSpeak.setField(1,t);
184      ThingSpeak.setField(2,h);
185      ThingSpeak.setField(3,Pa);
186      ThingSpeak.setField(4,mq135);
187      ThingSpeak.setField(5,conPM1);
188      ThingSpeak.setField(6,conPM25);
189
190      int x = ThingSpeak.writeFields(CHANNEL, WRITE_API);
191      if (x == 200) {Serial.println("Update realizado com sucesso");}
192      else {Serial.println("Problema no canal - erro HTTP " + String(x));}
193      prevMillisThingSpeak = millis();
194  }

```

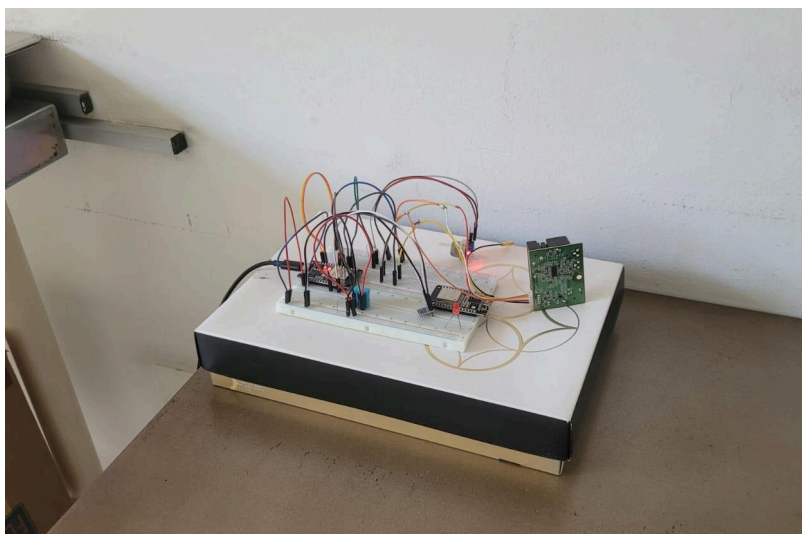
Fonte: De Autoria própria (2024).

3.3.3. Testes Propostos

Para a implementação dos testes, dois ambientes foram pensados para a execução destas experimentações. O primeiro teste foi realizado deixando os sensores medindo os valores em um ambiente residencial, um escritório, para ser mais exato. O outro teste foi realizado em um ambiente industrial, em uma área de serviço em uma padaria. Desta maneira, foi possível realizar as medições em dois tipos de ambiente diferentes durante cerca de 24 horas, para avaliar o desempenho destes sensores e a possibilidade de utilizá-los em conjunto com a plataforma ThingSpeak para o monitoramento destes parâmetros em diferentes aplicações, incluindo uma sala limpa.

O primeiro teste foi feito entre 22:41 do dia 27/07 e 22:29 do dia seguinte, no ambiente residencial. Já o segundo teste foi entre 14:50 do dia 29/07 até cerca de 14:00 de 30/07, no ambiente da Figura 40. Durante esse tempo, seria possível consultar os dados dos sensores por meio do ThingSpeak tanto pelo computador, quanto pelo celular, e isso foi importante para conseguir visualizar a operacionalidade do sistema enquanto o aparelho não estava à vista, no segundo teste, em particular.

Figura 40 - Circuito montado realizando medições no ambiente industrial



Fonte: De Autoria própria (2024).

4 RESULTADOS E DISCUSSÕES

4.1. Resultados Obtidos

Os dados referentes às medições foram coletados por meio dos sensores nas duas situações e ambientes propostos. De acordo com a Figura 41, a análise destes dados foi feita pela plataforma ThingSpeak, observando a curva dos valores referentes a cada um dos seis parâmetros presentes no sistema de monitoramento com o Esp32. Primeiramente, ao manter os sensores do sistema medindo as condições do ambiente residencial, foi possível analisar os valores encontrados ao longo das 24 horas com os sensores ligados e conectados à rede. A temperatura foi medida em graus Celsius ($^{\circ}\text{C}$), a umidade relativa do ar foi representada por porcentagem(%), a pressão ambiente está em hectopascal (hPa), os gases foram medidos em uma unidade adimensional ligada à tensão de saída do sensor e os sensores de partículas de poeira no ar medem em $\mu\text{g}/\text{m}^3$.

Figura 41 - Curva dos valores medidos no ThingSpeak no ambiente residencial

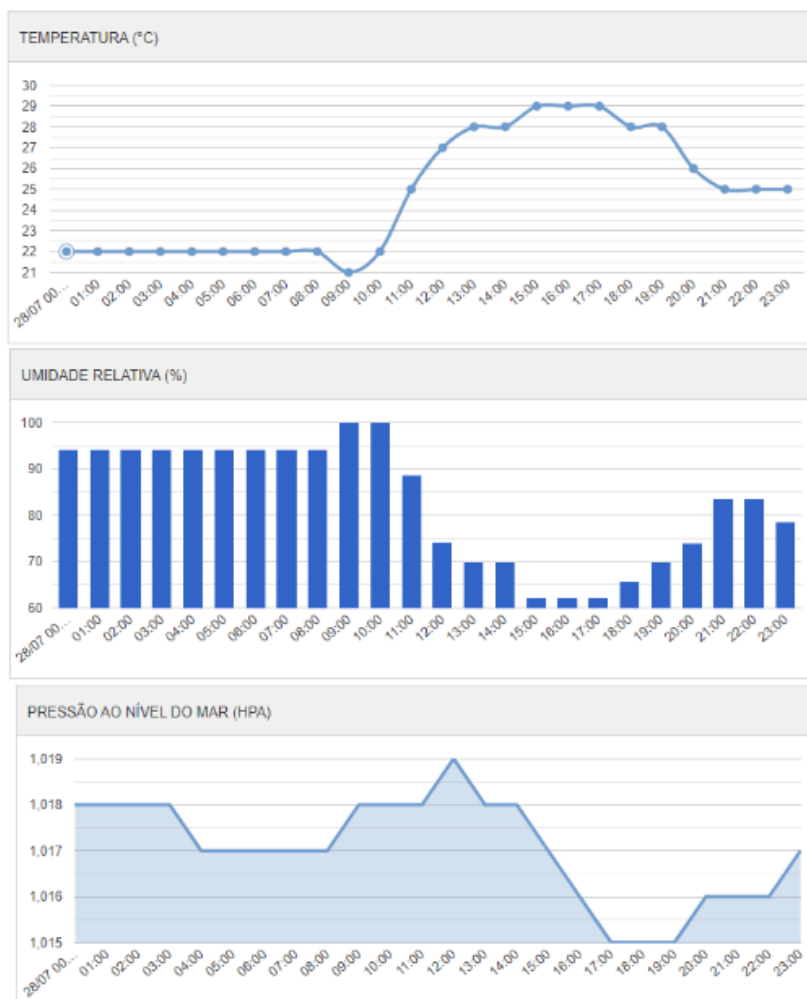


Fonte: De Autoria própria (2024).

Comparando valores de temperatura, umidade e pressão encontrados com os do site tempo.com, presentes na Figura 42, é possível ver uma similaridade nas curvas, as quais possuem um formato que acompanha as variações presentes nos gráficos da Figura 42. Além disso, é perceptível que o sensor de poeira de $PM_{1,0}$ e $PM_{2,5}$ possuem um padrão em que a concentração de partículas no ambiente mantêm-se entre 20 e 50 $\mu g/m^3$, com apenas alguns picos acima deste valor. Considerando que o ambiente doméstico tem seus parâmetros pouco controlados, os valores medidos a cerca de características locais seguem as variações

esperadas e podem ajudar a entender as condições na realização de atividades que dependem destes parâmetros.

Figura 42 - Gráficos de temperatura, umidade e pressão do dia 28/07/2024



Fonte: Tempo. 2024. Disponível em: <https://www.tempo.com/natal.htm>. Acesso em: 28 de julho de 2024.

Já as medições realizadas entre o dia 29/07 e 30/07 na área de serviço do ambiente industrial também possuem suas particularidades quanto à medição e se encontram na Figura 43. Os sensores também ficaram durante cerca de 24 horas medindo os parâmetros presentes no ambiente em questão. Os valores de temperatura, umidade e pressão comparados com os do site tempo.com, também é possível ver que a curva acompanha as variações do tempo presentes na Figura 44. Além disso, perto do horário de 6 horas da manhã, houve uma diminuição acentuada da temperatura e um aumento na umidade relativa. Isso pode ter ocorrido devido a

serviços no local da medição ligados à porta da câmara fria, a qual é localizada perto do sensor utilizado.

Figura 43 - Curva dos valores medidos no ThingSpeak no ambiente industrial

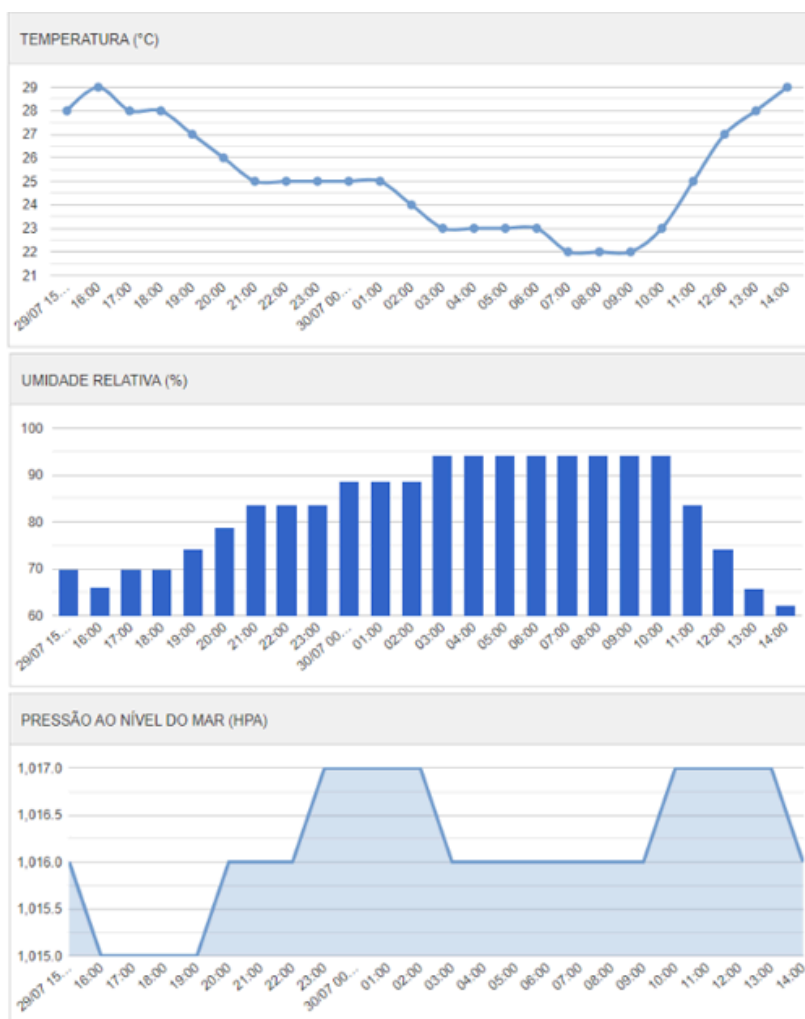


Fonte: De Autoria própria (2024).

Por meio dos valores de conversão de $\mu\text{g}/\text{m}^3$ para número de partículas/L, utilizando como referência os valores obtidos pelo $\text{PM}_{2.5}$, seria possível estimar a classificação de sala limpa conforme as convenções da ISO. Para o ambiente doméstico de escritório, considerando que a concentração se manteve abaixo de $50 \mu\text{g}/\text{m}^3$ durante a maior parte do experimento, a classificação seria de ISO-6. Ao

analisar os valores de concentração de gramas particulares do ar presentes no ambiente da área de serviço industrial, a qual manteve-se abaixo de $100 \mu\text{g}/\text{m}^3$, sua classificação também entra para a ISO-6.

Figura 44 - Gráficos de temperatura, umidade e pressão dos dias 29 e 30/07/2024



Fonte: Tempo. 2024. Disponível em: <https://www.tempo.com/natal.htm>. Acesso em: 30 de julho de 2024.

4.2. Discussões

Como foi citado anteriormente, fatores ambientais como a temperatura, pressão, umidade e partículas no ar têm a capacidade de interferir em atividades fundamentais de diversos tipos de processo. Tendo isso em vista, a utilização de um sistema de monitoramento dessas variáveis contribuiria para garantir a eficiência nos processos industriais e ajudaria com possíveis manutenções nos equipamentos, o

que seria mais vantajoso para evitar mais gastos com reposição e manutenções de emergência.

A atenção em relação a esses fenômenos precisa aumentar, pois o fato da cidade Natal se encontrar perto do mar e próxima da linha do Equador aumenta o fator da temperatura e umidade, os quais podem criar problemas nos maquinários à médio e longo prazo, caso não haja uma manutenção regular, prestando atenção a esses fatores. Além disso, o acúmulo de partículas nos equipamentos, principalmente em placas elétricas expostas, podem gerar curto-circuitos, acontecendo possíveis perdas materiais e até incêndios.

Tendo o conhecimento de que grupos de pesquisa e desenvolvimento da UFRN possuem o interesse em construir um ambiente de sala limpa, o sistema é uma alternativa interessante de baixo custo para ajudar com o monitoramento de alguns dos parâmetros no ambiente controlado. Considerando seu preço, pode-se dizer que o sistema é bem acessível comparando com outros no mercado e, dependendo de sua aplicação, é possível retirar alguns sensores para baratear o custo.

No contexto de salas limpas, o essencial é o sensor de detecção de poeira DSM501a, para o monitoramento das partículas no ar do ambiente, e o Esp32, para realizar a comunicação entre os sensores e a plataforma online. Os sensores de BMP180 e DHT11 também coletam dados importantes para várias aplicações em salas limpas e outros ambientes industriais, como já foi citado, e os dois possuem leitores de temperatura, utilizar os dois ajudaria na confiabilidade do sistema. E, tendo esse ponto em vista, o sensor MQ-135 seria mais útil para detecção de gases possivelmente perigosos. Além disso, por não abranger todos os tipos de gases, pode ser vantajoso escolher outros sensores da família MQ para outras aplicações específicas.

Tabela 2 - Custos para montagem do sistema de monitoramento

Equipamento	Custo
Módulo Esp32 Wi-fi + Bluetooth 30 pinos	R\$ 36,90
Sensor DHT11	R\$ 7,50
Sensor BMP180	R\$ 7,50
Sensor DSM501a	R\$ 69,90
Sensor MQ-135	R\$ 13,20
Conjunto de Jumpers	R\$ 5,60
Resistores	R\$ 1,40
Total	R\$ 142,00

Outro fator que pode ser levado em consideração é a aplicação do sistema para averiguar sobre as condições do ambiente em relação aos trabalhadores humanos. Pois, ao trabalhar com maquinários que podem expelir partículas, como fumaça e fuligem, a exposição prolongada pode gerar problemas de saúde para aqueles que realizam as operações. Para isso, os sensores de fumaça e partículas no ar ajudariam a manter o controle dos níveis destes parâmetros adversos, com o objetivo de ajudar a melhorar a qualidade de vida dos trabalhadores.

5 CONSIDERAÇÕES FINAIS

O desenvolvimento e implementação de sistemas de monitoramento em tempo real utilizando a tecnologia da Internet das Coisas (*IoT*) representam um avanço significativo para a gestão de ambientes industriais, especialmente nas áreas da saúde e nutrição, principalmente em contextos como o das salas limpas. A pesquisa conduzida ao longo deste trabalho demonstrou a viabilidade e os benefícios dessa abordagem, tanto em termos de eficiência operacional quanto de segurança.

Através da utilização de sensores inteligentes e microcontroladores, como o ESP32, foi possível monitorar parâmetros essenciais como temperatura, umidade, pressão e qualidade do ar. A integração desses sensores com plataformas de monitoramento *online*, como o *ThingSpeak*, permitiu não apenas o monitoramento em tempo real, mas também a análise detalhada dos dados coletados, facilitando a identificação de desvios, a tomada de decisões rápidas e informando os possíveis sistemas de controle que podem ser integrados ao monitoramento.

Os resultados obtidos destacaram a eficácia do sistema proposto, demonstrando que a automação e o monitoramento contínuo são cruciais para otimizar os processos industriais. A capacidade de prever problemas e responder rapidamente a eles reduz a necessidade de manutenções corretivas, resultando em economia de custos e aumento da produtividade. Além disso, a segurança dos trabalhadores é aprimorada ao minimizar os riscos de acidentes relacionados a falhas de comprometimento em relação à integridade dos materiais e equipamentos de trabalho.

Ao realizar os testes em ambientes diferentes, tendo uma noção de seus propósitos, analisando o sistema proposto com seus resultados e sabendo de suas limitações. É possível que esses equipamentos de baixo custo ajudem na construção de salas limpas, principalmente em ambiente de pesquisa e desenvolvimento. Pois, a programação e os equipamentos utilizados podem compor um sistema muito mais robusto, se melhor trabalhado, realizando testes na própria sala limpa a ser construída.

A continuidade deste trabalho envolve a realização de testes adicionais em diferentes contextos industriais e a expansão das funcionalidades do sistema. A automação da tomada de decisões com base nas métricas coletadas pelos sensores também é uma área promissora para futuros desenvolvimentos, permitindo uma resposta ainda mais eficiente a condições variáveis.

A implementação contínua de melhorias e a adaptação às necessidades específicas de cada contexto industrial são essenciais para maximizar os benefícios dessa abordagem tecnológica. Portanto, a implementação de um sistema de monitoramento robusto e acessível é fundamental para assegurar que as salas limpas atendam aos rígidos padrões de qualidade exigidos, minimizando os riscos de contaminação e garantindo a excelência dos produtos finais.

Em suma, a introdução de tecnologias de IoT e automação em ambientes industriais não só aumenta a eficiência e a segurança, mas também proporciona uma plataforma para inovações futuras, principalmente no contexto de salas limpas. Este trabalho não só demonstrou a viabilidade técnica e econômica do sistema proposto, mas também contribuiu para a evolução das práticas de monitoramento e gestão em ambientes industriais, no tocante das , em salas limpas.

6 REFERÊNCIAS BIBLIOGRÁFICAS

ALBUQUERQUE, Yure. **Esp32 Pinout - Guia Básico de GPIOs**. 2020. Disponível em: <https://blog.smartkits.com.br/esp32-pinout-guia-basico-de-gpios/>. Acesso em: 25 de junho de 2024.

ANANDAN, R.; GOPALAKRISHNAN, Suseendran; PAL, Souvik; ZAMAN, Noor. **Industrial Internet of Things (IIOT)**. [livro eletrônico]. Beverly. 2022.

ASMONTEC. **O que é uma sala limpa?** 2018. Disponível em: <https://asmontec.com.br/o-que-e-uma-sala-limpa/>. Acesso em: 07 de abril de 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). **NBR 13413: Controle de Contaminação**. [livro eletrônico]. Rio de Janeiro. 1995.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). **NBR ISO 14644-1: Salas limpas e ambientes controlados associados**. Rio de Janeiro. 2019. Disponível em: <https://www.normas.com.br/visualizar/abnt-nbr-nm/24244/abnt-nbriso14644-1-salas-limpas-e-ambientes-controlados-associados-parte-1-classificacao-da-limpeza-do-ar-por-concentracao-de-particulas>. Acesso em: 17 de julho de 2024.

BERTOLETI, Pedro. **Envie dados de temperatura e umidade relativa do ar para o ThingSpeak com o ESP8266**. 2020. Disponível em: <https://www.makerhero.com/blog/esp8266-com-thingspeak/>. Acesso em: 05 de julho de 2024.

BOSCH SENSORTEC GMBH. **BMP180 Digital Pressure Sensor**. [livro eletrônico]. Germany. 2013.

BRITTO, J. **Projeto Básico de Salas Limpas**. 2018. Disponível em: <https://www.adriferco.com.br/resources/Artigo%20SBCC%20-%20Ed%2054%20-%20Projeto%20B%C3%A1sico%20de%20Salas%20Limpas%20-%20Parte%201.pdf>. Acesso em: 29 de junho de 2024.

CATSOULIS, John. **Designing Embedded Hardware**. [livro eletrônico]. O'reilly. 2003.

ELECROW. **Dust Sensor- DSM501A**. Disponível em: <https://www.elecrow.com/wiki/dust-sensor--dsm501a.html>. Acesso em: 24 de julho de 2024.

2024.

ESPRESSIF. **ESP32 Series Datasheet**. 2024. Disponível em:

https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Acesso em: 04 de maio de 2024.

HANWEI ELECTRONICS CO., LTD. **Technical Data MQ-135 Gas Sensor**. “s.d.”

Disponível em: https://www.electronicoscaldas.com/datasheet/MQ-135_Hanwei.pdf.

Acesso em: 01 de julho de 2024.

INSTITUTO NACIONAL DE TECNOLOGIA E SAÚDE.(INTS). Procedimento

Operacional de controle de Temperatura e Umidade. [livro eletrônico]. Brasília. 2021.

KERSCHBAUMMER, Ricardo. **Microcontroladores**. Disponível em:

<https://professor.luzerna.ifc.edu.br/ricardo-kerschbaumer/wp-content/uploads/sites/43/2018/02/Apostila-Microcontroladores.pdf>. Santa Catarina. 2018.

MANO, Júlia; MARCOLINO, Aline. **Dispositivos IoT devem chegar a 27 bilhões até 2025**. 2022. Disponível em:

<https://www.poder360.com.br/brasil/dispositivos-iot-devem-chegar-a-27-bilhoes-ate-2025/>. Acesso em: 24 de abril de 2024.

MARCAS MAIS. **Tramontina inova e lança fogão conectado à internet**. 2021.

Disponível em:

https://marcasmais.com.br/minforma/noticias/marketing/tramontina-inova-e-lanca-fogao-conectado-a-internet/#google_vignette. Acesso em: 27 de abril de 2024.

MOUSER ELECTRONICS. **DHT11 Humidity & Temperature Sensor**. 2024.

Disponível em:

<https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf?srltid=AfmBOooTStNeFJQICFu8lLpAsAAiyqi9D98Wu5tP1INZFWTu8Fx8KljB>. Acesso em: 26 de junho de 2024.

OPENAI. **ChatGPT**. 2024. Disponível em: <https://www.chat.openai.com/chat>. Acesso em: 28 de julho de 2024.

ROSE, Karen; ELDRIDGE, Scott; CHAPIN, Lyman. **The Internet of Things: An Overview Understanding the Issues and Challenges of a More Connected World**. [livro eletrônico]. Internet Society. 2015.

ROY, Sandip; BOSE, Rajesh; SARDDAR, Debabrata. **Smart and Healthy City**

Protecting from Carcinogenic Pollutants. India. 2017. Disponível em:
https://www.ripublication.com/ijaes17/ijaesv12n9_04.pdf.

SAMYOUNG S&C CO., LTD. **Dust Sensor Module P/N: DSM501 Specifications.**
 [livro eletrônico]. Korea. 2014.

SEBRAE. **Os sensores nos processos industriais.** 2023. Disponível em:
<https://sebrae.com.br/sites/PortalSebrae/artigos/os-sensores-nos-processos-industriais,61fff6f145e96810VgnVCM1000001b00320aRCRD%23:~:text=Os%20sensores%20t%C3%AAm%20se%20tornado,de%20material%20dentro%20dos%20recipientes&sa=D&source=docs&ust=1722372018567491&usg=AOvVaw0RswxIKLZbVjP8uzjy1iz4>. Acesso em: 07 de abril de 2024.

VITAL, José; BRAGA, Eulina. **A logística no Transporte e Armazenamento de Medicamentos Termolábeis.** [livro eletrônico]. São Paulo. 2019.

WHL ENGENHARIA. **Incêndios em indústrias: causas, riscos e como evitar.** 2021. Disponível em:
<https://whlengenharia.com.br/incendios-em-industrias-causas-riscos-e-como-evitar/#:~:text=Inc%C3%AAndios%20em%20ind%C3%BAstrias%20s%C3%A3o%20ocorr%C3%AAncias,produ%C3%A7%C3%A3o%20de%20fa%C3%ADscas%2C%20por%20exemplo>. Acesso em: 11 de Abril de 2024.

7 APÊNDICE

APÊNDICE A – CÓDIGO COMPLETO DO SISTEMA

```

1  #include<string.h>
2  #define PM1PIN 35
3  #define PM25PIN 33
4  byte buff[2];
5  unsigned long durationPM1;
6  unsigned long durationPM25;
7  unsigned long starttime;
8  unsigned long endtime;
9  unsigned long samptime_ms = 30000;
10 unsigned long lowpulseoccupancyPM1 = 0;
11 unsigned long lowpulseoccupancyPM25 = 0;
12 int i=0;
13
14 #ifdef __cplusplus
15 extern "C" {
16 #endif
17 uint8_t temprature_sens_read();
18 #ifdef __cplusplus
19 }
20 #endif
21 uint8_t temprature_sens_read();
22
23 int intervalSensor = 2000;
24 long prevMillisThingSpeak = 0;
25 int intervalThingSpeak = 1440000; // (24 minutos)
26
27 #include <WiFi.h>
28 const char ssid[] = "*****";
29 const char password[] = "*****";
30 WiFiClient client;
31
32 #include <ThingSpeak.h>
33 const long CHANNEL = *****;
34 const char *WRITE_API = *****;
35
36 #include <Wire.h>
37 #include <Adafruit_BMP085.h>
38
39 Adafruit_BMP085 bmp;
40
41 #include <DHT.h> //CONFIGURAÇÃO DO DHT11
42 DHT dht(26, DHT11);
43
44 const int sensorMQ135 = 34;

```

```

46 void setup()
47 {
48     Serial.begin(115200);
49
50     if (!bmp.begin()) {
51         Serial.println("Could not find a valid BMP085/BMP180 sensor, check wiring!");
52         while (1) {}
53     }
54
55     dht.begin();
56
57     pinMode(PM1PIN, INPUT);
58     pinMode(PM25PIN, INPUT);
59     starttime = millis();
60
61     Serial.println();
62     Serial.println("Envia os dados do sensor para o ThingSpeak usando o ESP32");
63     Serial.println();
64
65     WiFi.mode(WIFI_STA); //Modo Station
66     ThingSpeak.begin(client); // Inicializa o ThingSpeak
67 }
68
69 float calculateConcentration(long lowpulseInMicroSeconds, long durationinSeconds){
70
71     float ratio = (lowpulseInMicroSeconds/1000000.0)/30.0*100.0;
72     float concentration = 1.915 * pow(ratio,2) + 95.22 * ratio - 48.84; // ug/m3
73     return concentration;
74 }
75
76 float t = 0;
77 float h = 0;
78 int mq135 = 0;
79 int mq2 = 0;
80 float Pa = 0;
81 float conPM1 = 0;
82 float conPM25 = 0;
83
84 void loop(){
85
86     t = dht.readTemperature();
87     h = dht.readHumidity();
88     Pa = bmp.readSealevelPressure();
89     Pa = Pa*(0.01);
90
91     mq135 = analogRead(sensorMQ135);
92     mq135 = map(mq135, 0, 4095, 0, 1000);
93
94     durationPM1 = pulseIn(PM1PIN, LOW);
95     durationPM25 = pulseIn(PM25PIN, LOW);

```

```

97 lowpulseoccupancyPM1 += durationPM1;
98 lowpulseoccupancyPM25 += durationPM25;
99
100 endtime = millis();
101 if ((endtime-starttime) > sampletime_ms)
102 {
103     conPM1 = calculateConcentration(lowpulseoccupancyPM1,30);
104     conPM25 = calculateConcentration(lowpulseoccupancyPM25,30);
105     lowpulseoccupancyPM1 = 0;
106     lowpulseoccupancyPM25 = 0;
107     starttime = millis();
108     Serial.print("PM1 ");
109     Serial.print(conPM1);
110     Serial.print(" ug/m3");
111     Serial.print(" PM25 ");
112     Serial.print(conPM25);
113     Serial.println(" ug/m3");
114 }
115
116
117 // Conecta ou reconecta o WiFi
118 if (WiFi.status() != WL_CONNECTED) {
119     Serial.print("Atenção para conectar o SSID: ");
120     Serial.println(ssid);
121     while (WiFi.status() != WL_CONNECTED) {
122         WiFi.begin(ssid, password);
123         Serial.print(".");
124         delay(5000);
125     }
126     Serial.println("\nConectado");
127 }
128
129 delay(1000);
130
131 if (millis() - prevMillisThingSpeak > intervalThingSpeak) {
132
133     // Configura os campos com os valores
134     ThingSpeak.setField(1,t);
135     ThingSpeak.setField(2,h);
136     ThingSpeak.setField(3,Pa);
137     ThingSpeak.setField(4,mq135);
138     ThingSpeak.setField(5,conPM1);
139     ThingSpeak.setField(6,conPM25);

```

```
141     int x = ThingSpeak.writeFields(CHANNEL, WRITE_API);
142     if (x == 200) {
143         Serial.println("Update realizado com sucesso");
144     }
145     else {
146
147         Serial.println("Problema no canal - erro HTTP " + String(x));
148     }
149
150     prevMillisThingSpeak = millis();
151 }
152 }
153
```